Distancias y caminos mínimos. Conectividad. Orientabilidad.

Matemática Discreta II

Curso 2025/26

Dpto. de Matemática Aplicada a las TIC
E.T.S. Ingenieros Informáticos
Universidad Politécnica de Madrid



UNIVERSIDAD POLITÉCNICA DE MADRID



ESCUELA TÉCNICA SUPERIOR DE INGENIEROS INFORMÁTICOS

Contenidos

1 Distancias en grafos

2 Caminos mínimos

3 Problemas de ubicación de servicios. Excentricidad, centro y diámetro.

4 Conectividad por vértices y aristas

5 Orientabilidad en grafos

Distancias en grafos

- Longitud de caminos: definición y ejemplos.
- Distancia entre vértices en grafos y digrafos.

Longitud de un camino

Definición y ejemplos

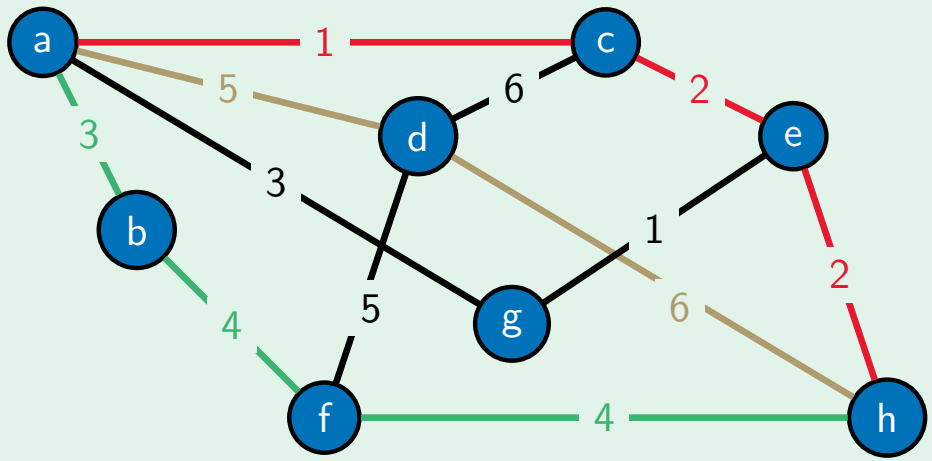
Definición

Sea $G = (V, E, \varphi, w)$ un grafo ponderado. Sean $u, v \in V$ dos vértices y P un camino de u a v. La **longitud** de P es la suma de los pesos de las aristas de P, es decir, el peso del subgrafo P.

Ejemplo (Longitud de caminos)

P_1, P_2, P_3
caminos de a hasta h

Longitud de P_1: $w(P_1) = 5$
Longitud de P_2: $w(P_2) = 11$
Longitud de P_2: $w(P_3) = 11$



Distancia entre dos vértices

Definición y ejemplos

Definición

Sea $G = (V, E, \varphi, w)$ un grafo ponderado. Dados dos vértices $u, v \in V$, la **distancia** (usual) de u a v es la mínima longitud de todos los caminos que van de u a v. Denotamos $d(u, v) = \text{mín}\{w(P) : P \text{ camino de } u \text{ a } v\}$.

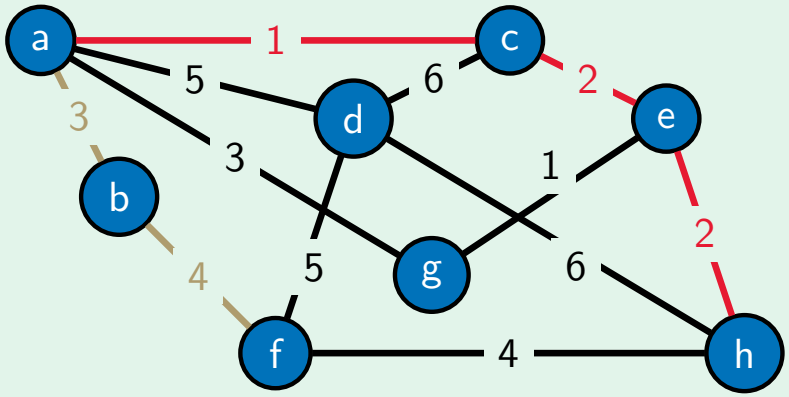
Convenciones

- Si no existe ningún camino entre u y v, escribimos $d(u, v) = \infty$.
- En un grafo sin pesos, definimos la distancia usual del mismo modo considerando que todas las aristas tienen peso 1.

Ejemplo (Distancia entre vértices)

$d(a, h) = 5$

$d(a, f) = 7$



Propiedades

Sea $G = (V, E, \varphi, w)$ un grafo con pesos positivos (tal que $w(e) > 0$ para toda arista $e \in E$). La distancia (usual) entre dos vértices verifica:

- **Positividad:**
  $d(u, v) \geq 0$ para todo $u, v \in V$ y $d(u, v) = 0 \iff u = v$.
- **Simetría:**
  $d(u, v) = d(v, u)$ para todo $u, v \in V$.
- **Desigualdad triangular:**
  $d(u, v) \leq d(u, z) + d(z, v)$ para todo $u, v, z \in V$.

Observación

Sea $G = (V, E, \varphi)$ un grafo (ponderado). Toda aplicación $D\colon V \times V \to \mathbb{R}$ que satisface las tres propiedades de la proposición anterior define una distancia entre vértices.

Distancia en digrafos

Definición y desambigüación

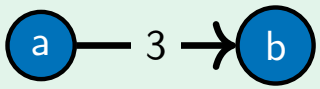
Definición

Sea $D = (V, E, \varphi, w)$ un digrafo ponderado. Dados dos vértices $u, v \in V$, la **distancia** (usual) de u a v es la mínima longitud de los caminos dirigidos que van desde u hasta v. Denotamos
$d(u, v) = \text{mín}\{w(P) : P \text{ camino dirigido de } u \text{ a } v\}$.

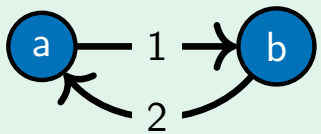
Observación

La definición anterior no cumple todas las propiedades para ser una "distancia".
En particular, la propiedad de simetría $d(u, v) = d(v, u)$ no es cierta en general

Ejemplo



$d(a, b) = 3$
$d(b, a) = \infty$



$d(a, b) = 1$
$d(b, a) = 2$

Caminos mínimos

- Algoritmo de Dijkstra: descripción y ejemplos.
- Algoritmo de Bellmann-Ford: descripción y ejemplos.
- Algoritmo de Floyd-Warshall: descripción y ejemplos.

Definición

Sea $G = (V, E, \varphi, w)$ un grafo con ponderado y sean $u, v \in V$ vértices. Un **camino mínimo** de u hasta v es un camino de longitud mínima, es decir, cuya longitud es la distancia $d(u, v)$.

Ejemplo (Longitud de caminos)

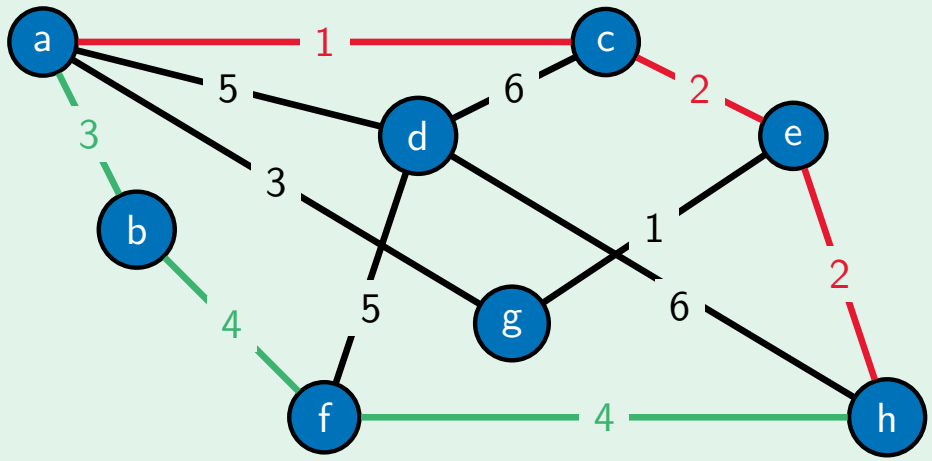
$$d(a, h) = 5$$

P_1, P_2 son caminos de a hasta h

$$w(P_1) = 5,\ w(P_2) = 11$$

P_1 es un camino mínimo

P_2 no es un camino mínimo



Algoritmo de Dijkstra

Objetivos

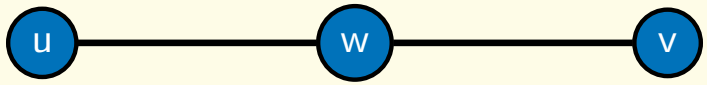
El algoritmo de Dijkstra sirve para...

- Construir un camino mínimo desde un vértice origen hasta el resto de vértices del grafo.
- Construir un camino mínimo desde un vértice origen hasta un vértice objetivo.

Fundamento

Sea $G = (V, E, \varphi, w)$ un grafo simple ponderado. Dados $u, v \in V$, queremos calcular $d(u, v)$ con su camino mínimo correspondiente.

Observamos que si w es un vértice interior de un camino C de longitud mínima de u a v, entonces C contiene un camino mínimo de u a w.



Descripción del algoritmo

Se introduce un grafo simple ponderado $G = (V, E, \varphi, w)$.

Se fija un vértice de origen $s \in V$ y un conjunto de vértices no visitados.

Se establece $D(s) = 0$ y $D(v) = \infty$ para los vértices no visitados.

En cada etapa

1. Se escoge el vértice u con menor valor $D(u)$ y se marca como visitado.
2. Se etiquetan los vértices con:
   - $D(v)$ es la longitud del camino mínimo de s hasta v.
   - $D(v) = \infty$, si no se encuentra un camino mínimo de s hasta v.
3. Se actualizan las etiquetas de los vértices adyacentes a u como

$$D(v) = \text{mín}\{D(v), D(u) + w(uv)\}$$

Se repiten en cada etapa hasta que todos los vértices han sido visitados o no quedan vértices alcanzables.

Al final, las etiquetas $D(u)$ indican la distancia mínima desde s hasta u.

Estructura del algoritmo

Input: Grafo (o digrafo) simple con pesos no negativos $G = (V, E, w)$ y vértice de origen $v_0 \in V$.

Clave: Construir un conjunto de vértices V_r para los que se conoce el camino más corto hasta v_0, hasta que $V_r = V$. Cada vértice v se asocia con la etiqueta $D(v)$ de longitud del camino encontrado.

Inicialización: $D(v_0) = d(v_0, v_0) = 0$, $D(v) = \infty$ para $v \neq v_0$, $V_r = \{\}$

Iteración: Elegir el vértice $u \notin V_r$ con etiqueta mínima. Añadir u a V_r. Analizar cada arista uv con $v \notin V_r$ y actualizar la etiqueta de v a $\text{mín}\{D(v), D(u) + w(uv)\}$. La iteración continúa hasta que $V_r = V$ o hasta que $d(v) = \infty$ para cada vértice $v \notin V_r$.

Output: Caminos mínimos desde v_0 hasta los otros vértices.

Pseudocódigo

$S \leftarrow V$ (no fijados); $V_r \leftarrow \emptyset$ (fijados); $v \leftarrow v_0$; $D \leftarrow [0, \infty, \ldots, \infty]$ (distancias);
$P \leftarrow [-1, 0, \ldots, 0]$ (predecesores) /* Inicialización de datos /*
while S no vacía

$v \leftarrow$ vértice de S con distancia $D(v)$ mínima

$S \leftarrow S - \{v\}$; $V_r \leftarrow V_r \cup \{v\}$ /* v pasa a ser fijado /*

Ad $\leftarrow \{$vértices de S adyacentes a $v\}$

for $u \in$Ad, hacer:

if $D(u) > D(v) + w(uv)$, **then**
$P(u) \leftarrow v$; $D(u) \leftarrow D(v) + w(uv)$ /* Actualización de predecesores y distancias /*
end if

end for

End while

Reconstruye *Caminos* usando P

Return $(Caminos, D)$ caminos y distancias desde v_0

Pseudocódigo

$S \leftarrow V$ (no fijados); $V_r \leftarrow \emptyset$ (fijados); $v \leftarrow v_0$ (origen); $F \leftarrow z$ (final);
$D \leftarrow [0, \infty, \ldots, \infty]$ (distancias); $P \leftarrow [-1, 0, \ldots, 0]$ (predecesores) /* Inicialización /*
while not encontrado F

$v \leftarrow$ vértice de S con distancia $D(v)$ mínima $S \leftarrow S - \{v\}$; $V_r \leftarrow V_r \cup \{v\}$

if $v \neq F$

Ad $\leftarrow \{$vértices de S adyacentes a $v\}$

for $u \in$Ad, hacer:

if $D(u) > D(v) + w(uv)$, **then**

$P(u) \leftarrow v$; $D(u) \leftarrow D(v) + w(uv)$ /* Actualización de predecesores y distancias /*

end if

end for

else encontrado F

end if

End while

Reconstruye *Camino de* v_0 *a* F usando P

Return (*Camino de* v_0 *a* F)

Algoritmo de Dijkstra con nodo final

Ejemplo

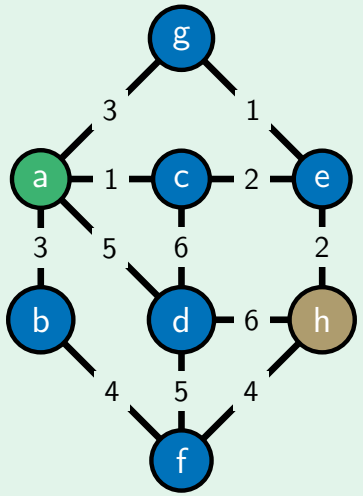
Ejemplo (Inicialización)

D

a	b	c	d	e	f	g	h
0	∞	∞	∞	∞	∞	∞	∞

P

a	b	c	d	e	f	g	h
-1	0	0	0	0	0	0	0



Origen: a ; Final: $F = h$; encontrado F = false

$V_r = \emptyset$, $S = \{a, b, c, d, e, f, g, h\}$

Definimos vértice final e inicial

Iniciamos los conjuntos de vértices fijados y no fijados

Iniciamos los vectores de distancias y predecesores

Pseudocódigo

$S \leftarrow V$; $V_r \leftarrow \emptyset$; $v \leftarrow v_0$; $F \leftarrow z$;

$D \leftarrow [0, \infty, \ldots, \infty]$; $P \leftarrow [-1, 0, \ldots, 0]$

/* Inicialización de datos /*

while not encontrado F

$v \leftarrow$ vértice en S con $D(v)$ mínima;

$S \leftarrow S - \{v\}$; $V_r \leftarrow V_r \cup \{v\}$

if $v \neq F$:

Ad $\leftarrow$ {vértices de S adyacentes a v}

for $u \in$Ad:

if $D(u) > D(v) + w(uv)$, **then**

$P(u) \leftarrow v$; $D(u) \leftarrow D(v) + w(uv)$

end if

end for

else encontrado F

end if

End while

Reconstruye *Camino de* v_0 *a* F usando P

Return (*Camino de* v_0 *a* F)

Algoritmo de Dijkstra con nodo final

Ejemplo

Ejemplo (Primera iteración)

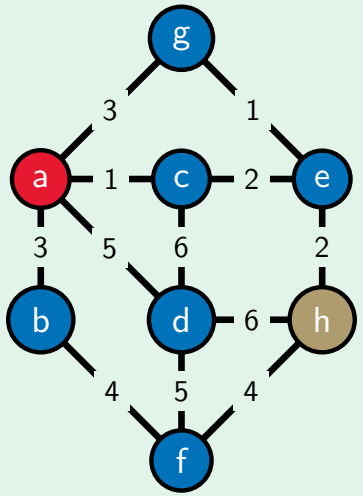
Comprobación y selección de vértice de distancia mínima

D

a	b	c	d	e	f	g	h
0	∞	∞	∞	∞	∞	∞	∞

P

a	b	c	d	e	f	g	h
-1	0	0	0	0	0	0	0



Origen: a ; Final: $F = h$; encontrado $F =$ false

$V_r = \emptyset$, $S = \{a, b, c, d, e, f, g, h\}$

Comprobamos la variable "encontrado F"

Seleccionamos el vértice de distancia D mínima: a

Pseudocódigo

$S \leftarrow V$; $V_r \leftarrow \emptyset$; $v \leftarrow v_0$; $F \leftarrow z$;
$D \leftarrow [0, \infty, \ldots, \infty]$; $P \leftarrow [-1, 0, \ldots, 0]$
/* Inicialización de datos /*

while not encontrado F

$v \leftarrow$ vértice en S con $D(v)$ mínima;

$S \leftarrow S - \{v\}$; $V_r \leftarrow V_r \cup \{v\}$
if $v \neq F$:

Ad $\leftarrow$ {vértices de S adyacentes a v}
for $u \in$Ad:
if $D(u) > D(v) + w(uv)$, **then**
$P(u) \leftarrow v$; $D(u) \leftarrow D(v) + w(uv)$
end if

end for

else encontrado F

end if

End while

Reconstruye *Camino de* v_0 *a* F usando P

Return (*Camino de* v_0 *a* F)

Algoritmo de Dijkstra con nodo final

Ejemplo

Ejemplo (Primera iteración)

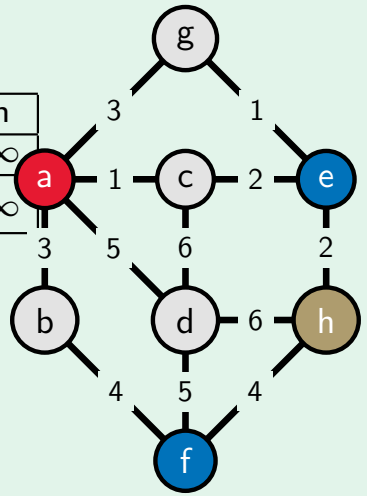
Actualización de estructuras

D

a	b	c	d	e	f	g	h
0	∞	∞	∞	∞	∞	∞	∞
0	**3**	**1**	**5**	∞	∞	**3**	∞

P

a	b	c	d	e	f	g	h
-1	0	0	0	0	0	0	0
-1	**a**	**a**	**a**	0	0	**a**	0



Origen: a ; Final: $F = h$; encontrado $F =$ false

$V_r = \{a\}$, $S = \{\cancel{a}, b, c, d, e, f, g, h\}$

Fijamos el vértice utilizado a

Identificamos los adyacentes de a en S: Ad $= \{b, c, d, g\}$

Actualizamos los vectores de distancias y predecesores

Pseudocódigo

$S \leftarrow V$; $V_r \leftarrow \emptyset$; $v \leftarrow v_0$; $F \leftarrow z$;
$D \leftarrow [0, \infty, \ldots, \infty]$; $P \leftarrow [-1, 0, \ldots, 0]$
/* Inicialización de datos /*

while not encontrado F

$v \leftarrow$ vértice en S con $D(v)$ mínima;

$S \leftarrow S - \{v\}$; $V_r \leftarrow V_r \cup \{v\}$

if $v \neq F$:

Ad $\leftarrow$ {vértices de S adyacentes a v}

for $u \in$Ad:

if $D(u) > D(v) + w(uv)$, **then**

$P(u) \leftarrow v$; $D(u) \leftarrow D(v) + w(uv)$

end if

end for

else encontrado F

end if

End while
Reconstruye *Camino de* v_0 *a* F usando P

Return (*Camino de* v_0 *a* F)

Ejemplo (Segunda iteración)

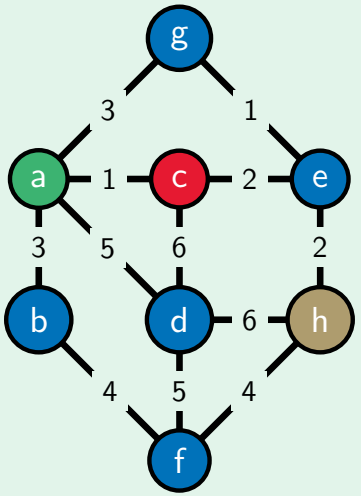
Comprobación y selección de vértice de distancia mínima

D

a	b	c	d	e	f	g	h
0	∞	∞	∞	∞	∞	∞	∞
0	3	1	5	∞	∞	3	∞

P

a	b	c	d	e	f	g	h
-1	0	0	0	0	0	0	0
-1	a	a	a	0	0	a	0



Origen: a ; Final: $F = h$; encontrado $F =$ false

$V_r = \{a\}$, $S = \{b, c, d, e, f, g, h\}$

Comprobamos la variable "encontrado F"

Seleccionamos el vértice de distancia D mínima: c

Pseudocódigo

$S \leftarrow V$; $V_r \leftarrow \emptyset$; $v \leftarrow v_0$; $F \leftarrow z$;
$D \leftarrow [0, \infty, \ldots, \infty]$; $P \leftarrow [-1, 0, \ldots, 0]$
/* Inicialización de datos /*

while not encontrado F

$v \leftarrow$ vértice en S con $D(v)$ mínima;

$S \leftarrow S - \{v\}$; $V_r \leftarrow V_r \cup \{v\}$
if $v \neq F$:

Ad $\leftarrow$ {vértices de S adyacentes a v}
for $u \in$Ad:
if $D(u) > D(v) + w(uv)$, **then**
$P(u) \leftarrow v$; $D(u) \leftarrow D(v) + w(uv)$
end if

end for

else encontrado F

end if

End while

Reconstruye *Camino de* v_0 *a* F usando P

Return (*Camino de* v_0 *a* F)

Algoritmo de Dijkstra con nodo final

Ejemplo

Ejemplo (Segunda iteración)

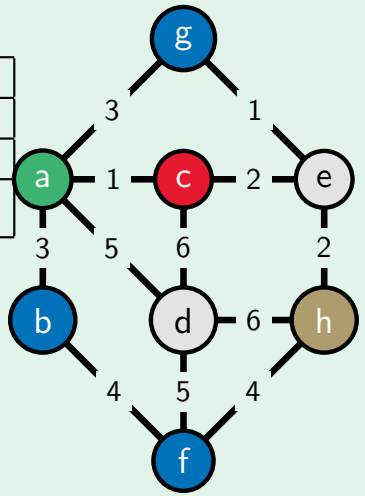
Actualización de estructuras

D

a	b	c	d	e	f	g	h
0	∞	∞	∞	∞	∞	∞	∞
0	3	1	5	∞	∞	3	∞
0	3	1	**5**	**3**	∞	3	∞

P

a	b	c	d	e	f	g	h
-1	0	0	0	0	0	0	0
-1	a	a	a	0	0	a	0
-1	a	a	**d**	**c**	0	a	0



Origen: a ; Final: $F = h$; encontrado $F =$ false

$V_r = \{a, c\}$, $S = \{b, \cancel{c}, d, e, f, g, h\}$

Fijamos el vértice utilizado c

Identificamos los adyacentes de c en S: Ad $= \{d, e\}$

Actualizamos los vectores de distancias y predecesores

Pseudocódigo

$S \leftarrow V$; $V_r \leftarrow \emptyset$; $v \leftarrow v_0$; $F \leftarrow z$;
$D \leftarrow [0, \infty, \ldots, \infty]$; $P \leftarrow [-1, 0, \ldots, 0]$
/* Inicialización de datos /*

while not encontrado F

$v \leftarrow$ vértice en S con $D(v)$ mínima;

$S \leftarrow S - \{v\}$; $V_r \leftarrow V_r \cup \{v\}$

if $v \neq F$:

Ad $\leftarrow$ {vértices de S adyacentes a v}

for $u \in$Ad:

if $D(u) > D(v) + w(uv)$, **then**

$P(u) \leftarrow v$; $D(u) \leftarrow D(v) + w(uv)$

end if

end for

else encontrado F

end if

End while

Reconstruye *Camino de* v_0 *a* F usando P

Return (*Camino de* v_0 *a* F)

Algoritmo de Dijkstra con nodo final

Ejemplo

Ejemplo (Tercera iteración)

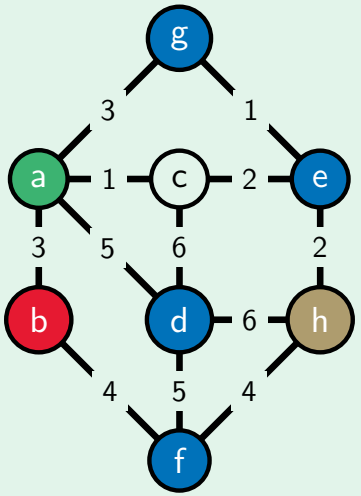
Comprobación y selección de vértice de distancia mínima

D

a	b	c	d	e	f	g	h
0	∞	∞	∞	∞	∞	∞	∞
0	3	1	5	∞	∞	3	∞
0	3	1	5	3	∞	3	∞

P

a	b	c	d	e	f	g	h
-1	0	0	0	0	0	0	0
-1	a	a	a	0	0	a	0
-1	a	a	d	c	0	a	0



Origen: a ; Final: $F = h$; encontrado $F =$ false

$V_r = \{a, c\}$, $S = \{b, d, e, f, g, h\}$

Comprobamos la variable "encontrado F"

Seleccionamos el vértice de distancia D mínima: b

Pseudocódigo

$S \leftarrow V$; $V_r \leftarrow \emptyset$; $v \leftarrow v_0$; $F \leftarrow z$;
$D \leftarrow [0, \infty, \ldots, \infty]$; $P \leftarrow [-1, 0, \ldots, 0]$
/* Inicialización de datos /*

while not encontrado F

$v \leftarrow$ vértice en S con $D(v)$ mínima;

$S \leftarrow S - \{v\}$; $V_r \leftarrow V_r \cup \{v\}$
if $v \neq F$:

Ad $\leftarrow$ {vértices de S adyacentes a v}
for $u \in$Ad:
if $D(u) > D(v) + w(uv)$, **then**
$P(u) \leftarrow v$; $D(u) \leftarrow D(v) + w(uv)$
end if

end for

else encontrado F

end if

End while

Reconstruye *Camino de* v_0 *a* F usando P

Return (*Camino de* v_0 *a* F)

Algoritmo de Dijkstra con nodo final

Ejemplo

Ejemplo (Tercera iteración)

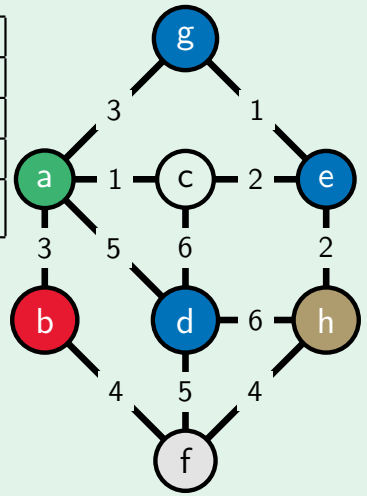
Actualización de estructuras

D

a	b	c	d	e	f	g	h
0	∞	∞	∞	∞	∞	∞	∞
0	3	1	5	∞	∞	3	∞
0	3	1	5	3	∞	3	∞
0	3	1	5	3	**7**	3	∞

P

a	b	c	d	e	f	g	h
-1	0	0	0	0	0	0	0
-1	a	a	a	0	0	a	0
-1	a	a	d	c	0	a	0
-1	a	a	d	c	**b**	a	0



Origen: a ; Final: $F = h$; encontrado F = false

$V_r = \{a, b, c\}$, $S = \{\not{b}, d, e, f, g, h\}$

Fijamos el vértice utilizado b

Identificamos los adyacentes de b en S: Ad $= \{f\}$

Actualizamos los vectores de distancias y predecesores

Pseudocódigo

$S \leftarrow V$; $V_r \leftarrow \emptyset$; $v \leftarrow v_0$; $F \leftarrow z$;
$D \leftarrow [0, \infty, \ldots, \infty]$; $P \leftarrow [-1, 0, \ldots, 0]$
/* Inicialización de datos /*

while not encontrado F

$v \leftarrow$ vértice en S con $D(v)$ mínima;

$S \leftarrow S - \{v\}$; $V_r \leftarrow V_r \cup \{v\}$

if $v \neq F$:

Ad $\leftarrow$ {vértices de S adyacentes a v}

for $u \in$Ad:

if $D(u) > D(v) + w(uv)$, **then**

$P(u) \leftarrow v$; $D(u) \leftarrow D(v) + w(uv)$

end if

end for

else encontrado F

end if

End while
Reconstruye *Camino de* v_0 *a* F usando P

Return (*Camino de* v_0 *a* F)

Algoritmo de Dijkstra con nodo final

Ejemplo

Ejemplo (Cuarta iteración)

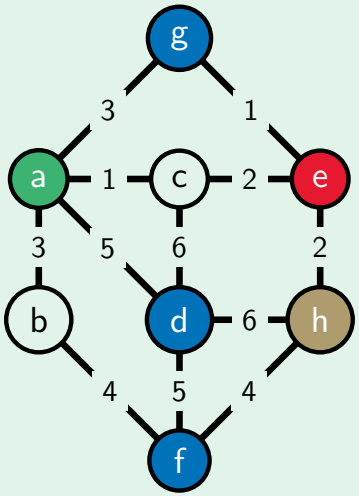
Comprobación y selección de vértice de distancia mínima

D

a	b	c	d	e	f	g	h
0	∞	∞	∞	∞	∞	∞	∞
0	3	1	5	∞	∞	3	∞
0	3	1	5	3	∞	3	∞
0	3	1	5	3	7	3	∞

P

a	b	c	d	e	f	g	h
-1	0	0	0	0	0	0	0
-1	a	a	a	0	0	a	0
-1	a	a	d	c	0	a	0
-1	a	a	d	c	b	a	0



Origen: a ; Final: $F = h$; encontrado $F =$ false

$V_r = \{a, b, c\}$, $S = \{d, e, f, g, h\}$

Comprobamos la variable "encontrado F"

Seleccionamos el vértice de distancia D mínima: e

Pseudocódigo

$S \leftarrow V$; $V_r \leftarrow \emptyset$; $v \leftarrow v_0$; $F \leftarrow z$;
$D \leftarrow [0, \infty, \ldots, \infty]$; $P \leftarrow [-1, 0, \ldots, 0]$
/* Inicialización de datos /*

while not encontrado F

$v \leftarrow$ vértice en S con $D(v)$ mínima;

$S \leftarrow S - \{v\}$; $V_r \leftarrow V_r \cup \{v\}$
if $v \neq F$:

Ad $\leftarrow$ {vértices de S adyacentes a v}
for $u \in$Ad:
if $D(u) > D(v) + w(uv)$, **then**
$P(u) \leftarrow v$; $D(u) \leftarrow D(v) + w(uv)$
end if

end for

else encontrado F

end if

End while

Reconstruye *Camino de* v_0 *a* F usando P

Return (*Camino de* v_0 *a* F)

Algoritmo de Dijkstra con nodo final

Ejemplo

Ejemplo (Cuarta iteración)

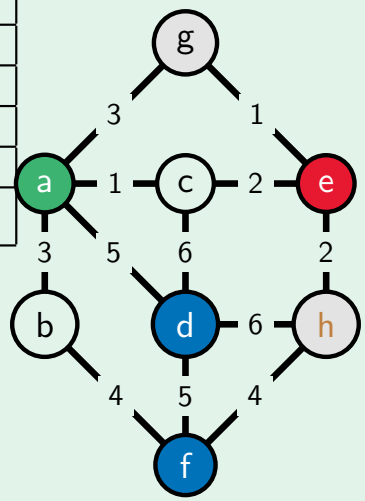
Actualización de estructuras

D

a	b	c	d	e	f	g	h
0	∞	∞	∞	∞	∞	∞	∞
0	3	1	5	∞	∞	3	∞
0	3	1	5	3	∞	3	∞
0	3	1	5	3	7	3	∞
0	3	1	5	3	7	**3**	**5**

P

a	b	c	d	e	f	g	h
-1	0	0	0	0	0	0	0
-1	a	a	a	0	0	a	0
-1	a	a	d	c	0	a	0
-1	a	a	d	c	b	a	0
-1	a	a	d	c	b	**a**	**e**



Origen: a ; Final: $F = h$; encontrado $F =$ false

$V_r = \{a, b, c, e\}$, $S = \{d, \cancel{e}, f, g, h\}$

Fijamos el vértice utilizado e

Identificamos los adyacentes de e en S: Ad $= \{g, h\}$

Actualizamos los vectores de distancias y predecesores

Pseudocódigo

$S \leftarrow V$; $V_r \leftarrow \emptyset$; $v \leftarrow v_0$; $F \leftarrow z$;
$D \leftarrow [0, \infty, \ldots, \infty]$; $P \leftarrow [-1, 0, \ldots, 0]$
/* Inicialización de datos /*

while not encontrado F

$v \leftarrow$ vértice en S con $D(v)$ mínima;

$S \leftarrow S - \{v\}$; $V_r \leftarrow V_r \cup \{v\}$

if $v \neq F$:

Ad $\leftarrow$ {vértices de S adyacentes a v}

for $u \in$Ad:

if $D(u) > D(v) + w(uv)$, **then**

$P(u) \leftarrow v$; $D(u) \leftarrow D(v) + w(uv)$

end if

end for

else encontrado F

end if

End while
Reconstruye *Camino de* v_0 *a* F usando P

Return (*Camino de* v_0 *a* F)

Algoritmo de Dijkstra con nodo final

Ejemplo

Ejemplo (Quinta iteración)

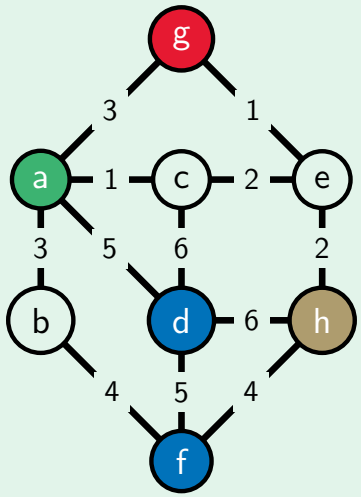
Comprobación y selección de vértice de distancia mínima

D

a	b	c	d	e	f	g	h
0	∞	∞	∞	∞	∞	∞	∞
0	3	1	5	∞	∞	3	∞
0	3	1	5	3	∞	3	∞
0	3	1	5	3	7	3	∞
0	3	1	5	3	7	3	5

P

a	b	c	d	e	f	g	h
-1	0	0	0	0	0	0	0
-1	a	a	a	0	0	a	0
-1	a	a	d	c	0	a	0
-1	a	a	d	c	b	a	0
-1	a	a	d	c	b	a	e



Origen: a ; Final: $F = h$; encontrado $F =$ false

$V_r = \{a, b, c, e\}$, $S = \{d, f, g, h\}$

Comprobamos la variable "encontrado F"

Seleccionamos el vértice de distancia D mínima: g

Pseudocódigo

$S \leftarrow V$; $V_r \leftarrow \emptyset$; $v \leftarrow v_0$; $F \leftarrow z$;
$D \leftarrow [0, \infty, \ldots, \infty]$; $P \leftarrow [-1, 0, \ldots, 0]$
/* Inicialización de datos /*

while not encontrado F

$v \leftarrow$ vértice en S con $D(v)$ mínima;

$S \leftarrow S - \{v\}$; $V_r \leftarrow V_r \cup \{v\}$
if $v \neq F$:

Ad $\leftarrow$ {vértices de S adyacentes a v}
for $u \in$Ad:
if $D(u) > D(v) + w(uv)$, **then**
$P(u) \leftarrow v$; $D(u) \leftarrow D(v) + w(uv)$
end if

end for

else encontrado F

end if

End while

Reconstruye *Camino de* v_0 *a* F usando P

Return (*Camino de* v_0 *a* F)

Algoritmo de Dijkstra con nodo final

Ejemplo

Ejemplo (Quinta iteración)

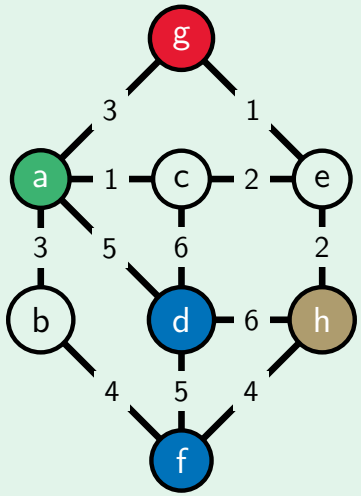
Actualización de estructuras

D

a	b	c	d	e	f	g	h
0	∞	∞	∞	∞	∞	∞	∞
0	3	1	5	∞	∞	3	∞
0	3	1	5	3	∞	3	∞
0	3	1	5	3	7	3	∞
0	3	1	5	3	7	3	5

P

a	b	c	d	e	f	g	h
-1	0	0	0	0	0	0	0
-1	a	a	a	0	0	a	0
-1	a	a	d	c	0	a	0
-1	a	a	d	c	b	a	0
-1	a	a	d	c	b	a	e



Origen: a ; Final: $F = h$; encontrado $F =$ false

$V_r = \{a, b, c, e, g\}$, $S = \{d, f, \cancel{g}, h\}$

Fijamos el vértice utilizado g

Identificamos los adyacentes de g en S: Ad $= \emptyset$

No actualizamos vectores porque no hay adyacentes

Pseudocódigo

$S \leftarrow V$; $V_r \leftarrow \emptyset$; $v \leftarrow v_0$; $F \leftarrow z$;
$D \leftarrow [0, \infty, \ldots, \infty]$; $P \leftarrow [-1, 0, \ldots, 0]$
/* Inicialización de datos /*

while not encontrado F

$v \leftarrow$ vértice en S con $D(v)$ mínima;

$S \leftarrow S - \{v\}$; $V_r \leftarrow V_r \cup \{v\}$

if $v \neq F$:

Ad $\leftarrow$ {vértices de S adyacentes a v}

for $u \in$Ad:

if $D(u) > D(v) + w(uv)$, **then**

$P(u) \leftarrow v$; $D(u) \leftarrow D(v) + w(uv)$

end if

end for

else encontrado F

end if

End while

Reconstruye *Camino de* v_0 *a* F usando P

Return (*Camino de* v_0 *a* F)

Algoritmo de Dijkstra con nodo final

Ejemplo

Ejemplo (Sexta iteración)

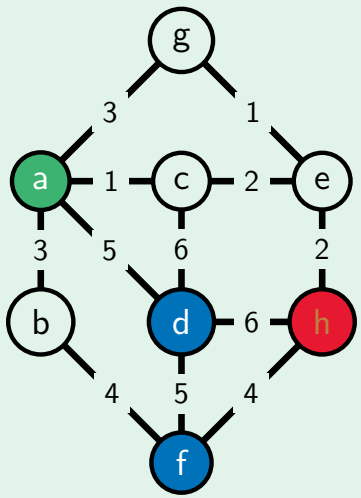
Comprobación y selección de vértice de distancia mínima

D

a	b	c	d	e	f	g	h
0	∞	∞	∞	∞	∞	∞	∞
0	3	1	5	∞	∞	3	∞
0	3	1	5	3	∞	3	∞
0	3	1	5	3	7	3	∞
0	3	1	5	3	7	3	5

P

a	b	c	d	e	f	g	h
-1	0	0	0	0	0	0	0
-1	a	a	a	0	0	a	0
-1	a	a	d	c	0	a	0
-1	a	a	d	c	b	a	0
-1	a	a	d	c	b	a	e



Origen: a ; Final: $F = h$; encontrado $F =$ false

$V_r = \{a, b, c, e, g\}$, $S = \{d, f, h\}$

Comprobamos la variable "encontrado F"

Seleccionamos el vértice de distancia D mínima: h

Pseudocódigo

$S \leftarrow V$; $V_r \leftarrow \emptyset$; $v \leftarrow v_0$; $F \leftarrow z$;
$D \leftarrow [0, \infty, \ldots, \infty]$; $P \leftarrow [-1, 0, \ldots, 0]$
/* Inicialización de datos /*

while not encontrado F

$v \leftarrow$ vértice en S con $D(v)$ mínima;

$S \leftarrow S - \{v\}$; $V_r \leftarrow V_r \cup \{v\}$
if $v \neq F$:

Ad $\leftarrow$ {vértices de S adyacentes a v}
for $u \in$Ad:
if $D(u) > D(v) + w(uv)$, **then**
$P(u) \leftarrow v$; $D(u) \leftarrow D(v) + w(uv)$
end if

end for

else encontrado F

end if

End while

Reconstruye *Camino de* v_0 *a* F usando P

Return (*Camino de* v_0 *a* F)

Algoritmo de Dijkstra con nodo final

Ejemplo

Ejemplo (Sexta iteración)

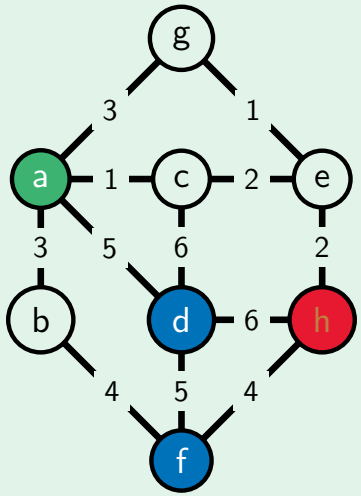
Actualización de estructuras

D

a	b	c	d	e	f	g	h
0	∞	∞	∞	∞	∞	∞	∞
0	3	1	5	∞	∞	3	∞
0	3	1	5	3	∞	3	∞
0	3	1	5	3	7	3	∞
0	3	1	5	3	7	3	5

P

a	b	c	d	e	f	g	h
-1	0	0	0	0	0	0	0
-1	a	a	a	0	0	a	0
-1	a	a	d	c	0	a	0
-1	a	a	d	c	b	a	0
-1	a	a	d	c	b	a	e



Origen: a ; Final: $F = h$; encontrado F = true

$V_r = \{a, b, c, e, g, h\}$, $S = \{d, f, h\}$

Fijamos el vértice utilizado h

Observamos que $h = F$ es el vértice final

Actualizamos la variable de vértice final encontrado

Pseudocódigo

$S \leftarrow V$; $V_r \leftarrow \emptyset$; $v \leftarrow v_0$; $F \leftarrow z$;
$D \leftarrow [0, \infty, \ldots, \infty]$; $P \leftarrow [-1, 0, \ldots, 0]$
/* Inicialización de datos /*

while not encontrado F

$v \leftarrow$ vértice en S con $D(v)$ mínima;

$S \leftarrow S - \{v\}$; $V_r \leftarrow V_r \cup \{v\}$

if $v \neq F$:

Ad $\leftarrow$ {vértices de S adyacentes a v}
for $u \in$Ad:
if $D(u) > D(v) + w(uv)$, **then**
$P(u) \leftarrow v$; $D(u) \leftarrow D(v) + w(uv)$
end if

end for

else encontrado F

end if

End while
Reconstruye *Camino de* v_0 *a* F usando P
Return (*Camino de* v_0 *a* F)

Ejemplo (Séptima iteración)

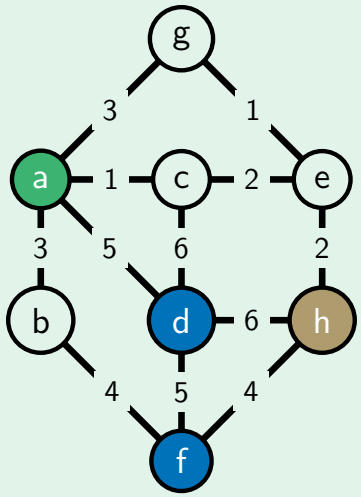
Comprobación y reconstrucción del camino

D

a	b	c	d	e	f	g	h
0	∞	∞	∞	∞	∞	∞	∞
0	3	1	5	∞	∞	3	∞
0	3	1	5	3	∞	3	∞
0	3	1	5	3	7	3	∞
0	3	1	5	3	7	3	5

P

a	b	c	d	e	f	g	h
-1	0	0	0	0	0	0	0
-1	a	a	a	0	0	a	0
-1	a	a	d	c	0	a	0
-1	a	a	d	c	b	a	0
-1	a	a	d	c	b	a	e



Origen: a ; Final: $F = h$; encontrado F = true

$V_r = \{a, b, c, e, g, h\}$, $S = \{d, f\}$

Comprobamos la variable "encontrado F"

Reconstruimos el camino de a hasta h usando los predecesores

$P(h) = e$, $P(e) = c$, $P(c) = a$, $P(a) = -1$

Pseudocódigo

$S \leftarrow V$; $V_r \leftarrow \emptyset$; $v \leftarrow v_0$; $F \leftarrow z$;
$D \leftarrow [0, \infty, \ldots, \infty]$; $P \leftarrow [-1, 0, \ldots, 0]$
/* Inicialización de datos /*

while not encontrado F

$v \leftarrow$ vértice en S con $D(v)$ mínima;
$S \leftarrow S - \{v\}$; $V_r \leftarrow V_r \cup \{v\}$
if $v \neq F$:

Ad $\leftarrow$ {vértices de S adyacentes a v}
for $u \in$Ad:
if $D(u) > D(v) + w(uv)$, **then**
$P(u) \leftarrow v$; $D(u) \leftarrow D(v) + w(uv)$
end if

end for

else encontrado F

end if

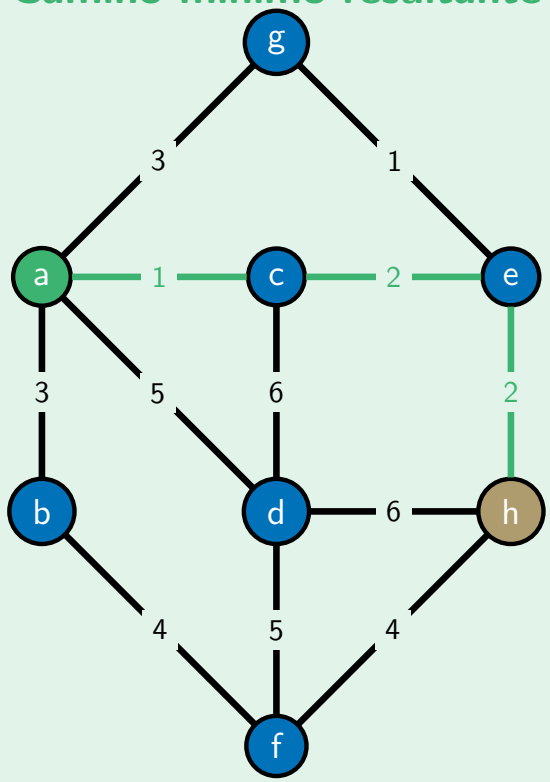
End while

Reconstruye *Camino de* v_0 *a* F usando P

Return (*Camino de* v_0 *a* F)

Ejemplo (Resultado)

Camino mínimo resultante



Camino de a hasta h: $P = a \mapsto c \mapsto e \mapsto h$

Peso mínimo: $w(P) = 5$

Pseudocódigo

$S \leftarrow V$; $V_r \leftarrow \emptyset$; $v \leftarrow v_0$; $F \leftarrow z$;
$D \leftarrow [0, \infty, \ldots, \infty]$; $P \leftarrow [-1, 0, \ldots, 0]$
/* Inicialización de datos /*

while not encontrado F

$v \leftarrow$ vértice en S con $D(v)$ mínima;
$S \leftarrow S - \{v\}$; $V_r \leftarrow V_r \cup \{v\}$
if $v \neq F$:

Ad $\leftarrow$ {vértices de S adyacentes a v}
for $u \in$Ad:
if $D(u) > D(v) + w(uv)$, **then**
$P(u) \leftarrow v$; $D(u) \leftarrow D(v) + w(uv)$
end if

end for

else encontrado F

end if

End while
Reconstruye *Camino de* v_0 *a* F usando P
Return (*Camino de* v_0 *a* F)

Algoritmo de Dijkstra

Teorema de validez

Teorema: Validez del algoritmo de Dijkstra

Sea $G = (V, E, w)$ un grafo simple con pesos no negativos. Dado un vértice $s \in V$, el algoritmo de Dijkstra calcula $d(s, z)$ para cada $z \in V$.

Demostración.

El algoritmo devuelve una etiqueta $D(z)$ para cada vértice $z \in V$.
Debemos probar que $D(z) = d(s, z)$ es la distancia mínima.
Sean $v_0, v_1, \ldots, v_{n-1}$ los vértices de G ordenados por su incorporación al conjunto V_r de vértices fijados, siendo v_0 el vértice inicial.
Procedemos por inducción sobre n, el número de vértices:

1. Es cierto para $n = 1$, pues v_0 es el inicio y $d(v_0, v_0) = 0 = D(v_0)$.
2. Hipótesis de inducción. Suponemos $D(v_1) = d(s, v_1), \ldots, D(v_k) = d(s, v_k)$.
3. Debemos probar que $D(v_{k+1}) = d(s, v_{k+1})$

Continuación de la demostración.

Probamos que $D(x_k) = d(s, v_{k+1})$.

- En este paso del algoritmo, $V_r = \{v_0, v_1, \ldots, v_k\}$ y la etiqueta $D(v_{k+1})$ es la longitud de un camino P que empieza en v_0, recorre diferentes vértices hasta llegar a v_k y después conecta ese vértice con una arista hasta v_{k+1}.
- Debemos comprobar que la longitud de P es mínima. Es decir, que si existe otro camino P' de v_0 hasta v_{k+1}, entonces $w(P) \leq w(P')$.
  - Sea z el primer vértice de P' que no está en V_r, vz la arista de P' que llega hasta z y P'' el resto del camino desde z hasta v_{k+1}.
  - Se tiene: $w(P') = d(v_0, v) + w(vz) + w(P'') \geq D(z) + w(P'')$.
  - Como v_{k+1} se elige como vértice de menor etiqueta: $D(z) \geq D(v_{k+1})$
  - Entonces: $w(P') \geq D(v_{k+1}) + w(P'') \geq D(v_{k+1}) = w(P)$
- Por tanto, P es un camino de peso mínimo y $D(v_{k+1}) = w(P) = d(v_0, v_{k+1})$

Algoritmo de Bellmann-Ford

Estructura del algoritmo

Input: Grafo (o digrafo) simple con pesos $G = (V, E, w)$ y vértice de origen $v_0 \in V$.

Clave: Considerar una a una todas las aristas intentando mejorar en cada paso las etiquetas $D(v)$ de los vértices.

Inicialización: $D(v_0) = d(v_0, v_0) = 0$, $D(v) = \infty$ para $v \neq v_0$

Iteración: Se considera cada arista uv del grafo. Si la distancia $D(v)$ puede ser mejorada con la arista uv, es decir, $D(v) > D(u) + w(uv)$, entonces actualizamos la etiqueta de v a $D(v) = D(u) + w(uv)$. El proceso de iteración continúa hasta que no se producen cambios respecto a la iteración anterior o se han realizado $n - 1$ iteraciones.

Output: Distancia y caminos mínimos desde v_0 hasta los otros vértices.

Pseudocódigo: Inicio y relajación de aristas

$n \leftarrow |V|$; $D \leftarrow [0, \infty, \ldots, \infty]$ (distancias); $P \leftarrow [-1, 0, \ldots, 0]$ (predecesores);
$i \leftarrow 0$ /* Inicialización de datos /*
while $i < n - 1$ and D modificado en la última iteración
    for $uv \in E$, hacer: /* Para cada arista del grafo /*
        $u \leftarrow$ origen de uv; $v \leftarrow$ extremo de uv;
        if $D(v) > D(u) + w(uv)$, **then**
        $P(v) \leftarrow u$; $D(v) \leftarrow D(u) + w(uv)$ /* Actualización de predecesores y distancias /*
        end if
    end for
    $i \leftarrow i + 1$
End while
Detección de ciclo de peso negativo /* Continúa en la siguiente página /*

Pseudocódigo: Detección de ciclos negativos

if $i < n-1$ or D no modificado en la última iteración /* Casos donde no hay ciclos negativos /*
    Reconstruye *Caminos* utilizando P /* No hay ciclos negativos /*
else
    $control \leftarrow$ correcto
    while uv arista no comprobada and $control \neq$ error
        $u \leftarrow$ origen de uv; $v \leftarrow$ extremo de uv;
        if $D(v) > D(u) + w(uv)$, **then**
            $control \leftarrow$ error
        end if
    end while
    if $control =$ error
        Return "Hay un ciclo de peso negativo"
    else Reconstruye *Caminos* utilizando P
    end if
End if
Return (*Caminos desde* v_0)

Algoritmo de Bellmann-Ford

Ejemplo sin ciclos negativos

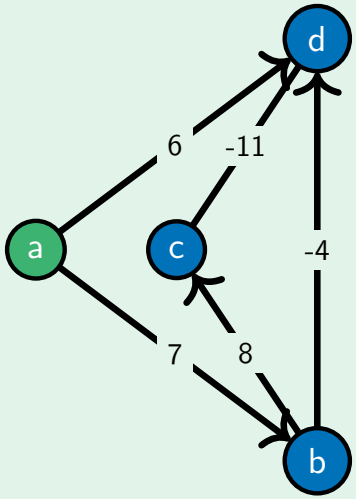
Ejemplo (Inicialización)

D

a	b	c	d
0	∞	∞	∞

P

a	b	c	d
-1	0	0	0



Origen: a ; $i = 0$; $n = 4$

Señalamos el vértice inicial v_0

Iniciamos el contador de iteraciones

Iniciamos los vectores de distancias y predecesores

Pseudocódigo

$n \leftarrow |V|$; $i \leftarrow 0$; $D \leftarrow [0, \infty, \ldots, \infty]$;

$P \leftarrow [-1, 0, \ldots, 0]$ /* Inicialización de datos /*

while $i < n - 1$ and D modificado en última iteración

for $uv \in E$: /* Para cada arista del grafo /*

$u \leftarrow$ origen de uv; $v \leftarrow$ extremo de uv;
if $D(v) > D(u) + w(uv)$, **then**
$P(v) \leftarrow u$; $D(v) \leftarrow D(u) + w(uv)$
end if

end for

$i \leftarrow i + 1$

End while

Detección de ciclo de peso negativo

Ejemplo (Primera iteración)

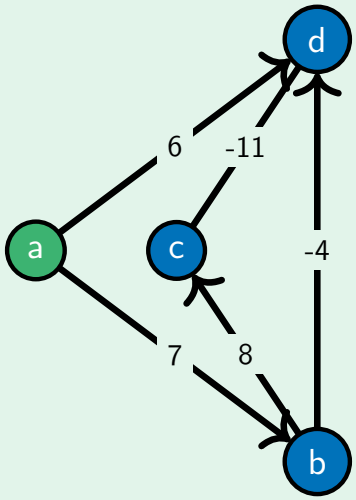
Comprobación del paso

D

a	b	c	d
0	∞	∞	∞

P

a	b	c	d
-1	0	0	0



Origen: a ; $i = 0$; $n = 4$

Comprobamos que $i < 3$ y D ha sido modificado al iniciar

Comenzamos la revisión de aristas

Pseudocódigo

$n \leftarrow |V|$; $i \leftarrow 0$; $D \leftarrow [0, \infty, \ldots, \infty]$;
$P \leftarrow [-1, 0, \ldots, 0]$ /* Inicialización de datos /*

while $i < n - 1$ and D modificado en última iteración

for $uv \in E$: /* Para cada arista del grafo /*
$u \leftarrow$ origen de uv; $v \leftarrow$ extremo de uv;
if $D(v) > D(u) + w(uv)$, **then**
$P(v) \leftarrow u$; $D(v) \leftarrow D(u) + w(uv)$
end if

end for

$i \leftarrow i + 1$

End while

Detección de ciclo de peso negativo

Algoritmo de Bellmann-Ford

Ejemplo sin ciclos negativos

Ejemplo (Primera iteración)

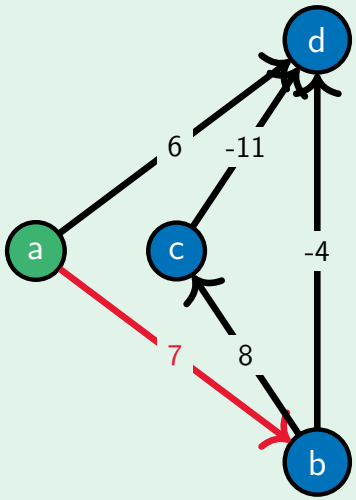
Relajación de aristas

D

a	b	c	d
0	∞	∞	∞
0	7	∞	∞

P

a	b	c	d
-1	0	0	0
-1	a	0	0



Origen: a ; $i = 0$; $n = 4$

Seleccionamos la arista ab

Esta arista mejora la distancia a b

Actualizamos distancia y predecesor de b

Pseudocódigo

$n \leftarrow |V|$; $i \leftarrow 0$; $D \leftarrow [0, \infty, \ldots, \infty]$;
$P \leftarrow [-1, 0, \ldots, 0]$ /* Inicialización de datos /*

while $i < n - 1$ and D modificado en última iteración

for $uv \in E$: /* Para cada arista del grafo /*

$u \leftarrow$ origen de uv; $v \leftarrow$ extremo de uv;

if $D(v) > D(u) + w(uv)$, **then**

$P(v) \leftarrow u$; $D(v) \leftarrow D(u) + w(uv)$

end if

end for

$i \leftarrow i + 1$

End while

Detección de ciclo de peso negativo

Algoritmo de Bellmann-Ford

Ejemplo sin ciclos negativos

Ejemplo (Primera iteración)

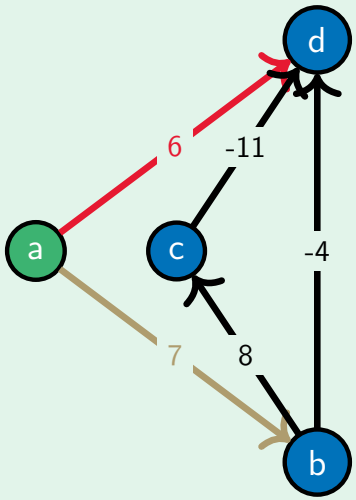
Relajación de aristas

D

a	b	c	d
0	∞	∞	∞
0	7	∞	∞
0	7	∞	6

P

a	b	c	d
-1	0	0	0
-1	a	0	0
-1	a	0	a



Origen: a ; $i = 0$; $n = 4$

Seleccionamos la arista ad

Esta arista mejora la distancia a d

Actualizamos distancia y predecesor de d

Pseudocódigo

$n \leftarrow |V|$; $i \leftarrow 0$; $D \leftarrow [0, \infty, \ldots, \infty]$;
$P \leftarrow [-1, 0, \ldots, 0]$ /* Inicialización de datos /*

while $i < n - 1$ and D modificado en última iteración

for $uv \in E$: /* Para cada arista del grafo /*

$u \leftarrow$ origen de uv; $v \leftarrow$ extremo de uv;

if $D(v) > D(u) + w(uv)$, **then**

$P(v) \leftarrow u$; $D(v) \leftarrow D(u) + w(uv)$

end if

end for

$i \leftarrow i + 1$

End while

Detección de ciclo de peso negativo

Algoritmo de Bellmann-Ford

Ejemplo sin ciclos negativos

Ejemplo (Primera iteración)

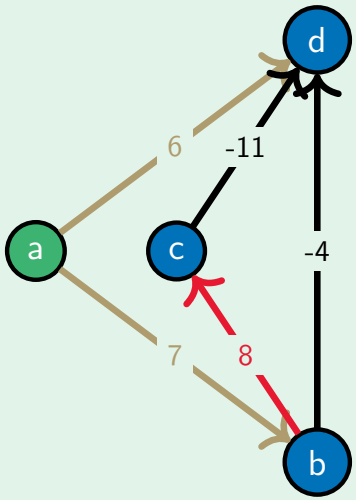
D

a	b	c	d
0	∞	∞	∞
0	7	∞	∞
0	7	∞	6
0	7	15	6

P

a	b	c	d
-1	0	0	0
-1	a	0	0
-1	a	0	a
-1	a	b	a

Relajación de aristas



Origen: a ; $i = 0$; $n = 4$

Seleccionamos la arista bc

Esta arista mejora la distancia a c

Actualizamos distancia y predecesor de c

Pseudocódigo

$n \leftarrow |V|$; $i \leftarrow 0$; $D \leftarrow [0, \infty, \ldots, \infty]$;
$P \leftarrow [-1, 0, \ldots, 0]$ /* Inicialización de datos /*

while $i < n - 1$ and D modificado en última iteración

for $uv \in E$: /* Para cada arista del grafo /*

$u \leftarrow$ origen de uv; $v \leftarrow$ extremo de uv;

if $D(v) > D(u) + w(uv)$, **then**

$P(v) \leftarrow u$; $D(v) \leftarrow D(u) + w(uv)$

end if

end for

$i \leftarrow i + 1$

End while

Detección de ciclo de peso negativo

Ejemplo (Primera iteración)

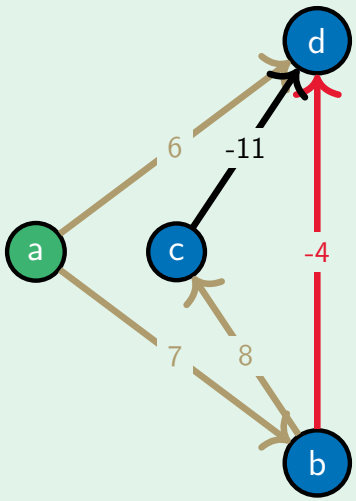
Relajación de aristas

D

a	b	c	d
0	∞	∞	∞
0	7	∞	∞
0	7	15	∞
0	7	15	6
0	7	15	3

P

a	b	c	d
-1	0	0	0
-1	a	0	0
-1	a	0	a
-1	a	b	a
-1	a	b	b



Origen: a ; $i = 0$; $n = 4$

Seleccionamos la arista bd

Esta arista mejora la distancia a d

Actualizamos distancia y predecesor de d

Pseudocódigo

$n \leftarrow |V|$; $i \leftarrow 0$; $D \leftarrow [0, \infty, \ldots, \infty]$;
$P \leftarrow [-1, 0, \ldots, 0]$ /* Inicialización de datos /*

while $i < n - 1$ and D modificado en última iteración

for $uv \in E$: /* Para cada arista del grafo /*

$u \leftarrow$ origen de uv; $v \leftarrow$ extremo de uv;

if $D(v) > D(u) + w(uv)$, **then**

$P(v) \leftarrow u$; $D(v) \leftarrow D(u) + w(uv)$

end if

end for

$i \leftarrow i + 1$

End while

Detección de ciclo de peso negativo

Algoritmo de Bellmann-Ford

Ejemplo sin ciclos negativos

Ejemplo (Primera iteración)

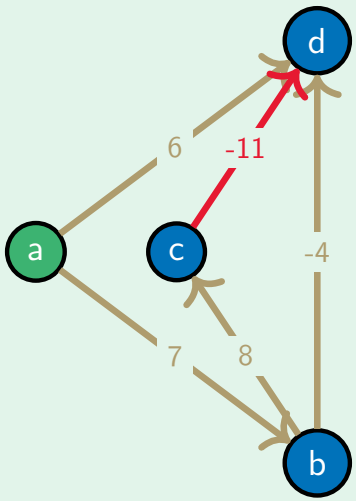
Relajación de aristas

D

a	b	c	d
0	∞	∞	∞
0	7	∞	∞
0	7	15	∞
0	7	15	6
0	7	15	3

P

a	b	c	d
-1	0	0	0
-1	a	0	0
-1	a	0	a
-1	a	b	a
-1	a	b	b



Origen: a ; $i = 0$; $n = 4$

Seleccionamos la arista cd

Esta arista no mejora la distancia a d

No actualizamos distancia ni predecesor de d

Pseudocódigo

$n \leftarrow |V|$; $i \leftarrow 0$; $D \leftarrow [0, \infty, \ldots, \infty]$;
$P \leftarrow [-1, 0, \ldots, 0]$ /* Inicialización de datos /*

while $i < n - 1$ and D modificado en última iteración

for $uv \in E$: /* Para cada arista del grafo /*

$u \leftarrow$ origen de uv; $v \leftarrow$ extremo de uv;

if $D(v) > D(u) + w(uv)$, **then**

$P(v) \leftarrow u$; $D(v) \leftarrow D(u) + w(uv)$

end if

end for

$i \leftarrow i + 1$

End while

Detección de ciclo de peso negativo

Ejemplo (Primera iteración)

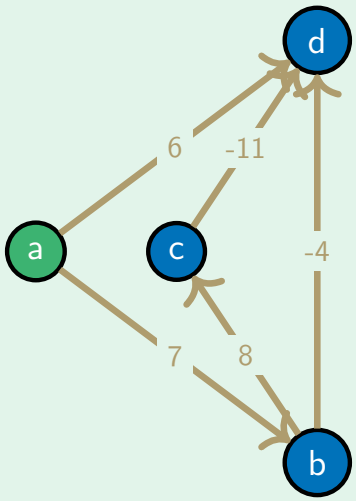
Actualización del contador

D

a	b	c	d
0	∞	∞	∞
0	7	∞	∞
0	7	15	∞
0	7	15	6
0	7	15	3

P

a	b	c	d
-1	0	0	0
-1	a	0	0
-1	a	0	a
-1	a	b	a
-1	a	b	b



Origen: a ; $i = 0 + 1$; $n = 4$

No quedan más aristas

Incrementamos el contador

Pseudocódigo

$n \leftarrow |V|$; $i \leftarrow 0$; $D \leftarrow [0, \infty, \ldots, \infty]$;
$P \leftarrow [-1, 0, \ldots, 0]$ /* Inicialización de datos /*

while $i < n - 1$ and D modificado en última iteración

for $uv \in E$: /* Para cada arista del grafo /*

$u \leftarrow$ origen de uv; $v \leftarrow$ extremo de uv;
if $D(v) > D(u) + w(uv)$, **then**
$P(v) \leftarrow u$; $D(v) \leftarrow D(u) + w(uv)$
end if

end for

$i \leftarrow i + 1$

End while

Detección de ciclo de peso negativo

Ejemplo (Segunda iteración)

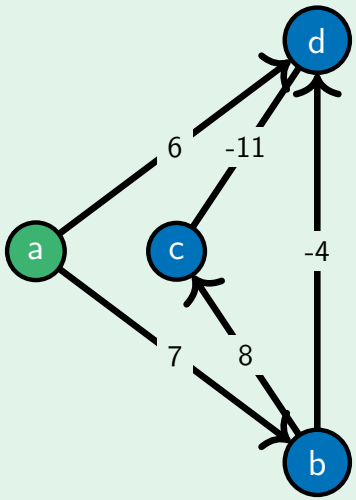
Comprobación del paso

D

a	b	c	d
0	∞	∞	∞
0	7	∞	∞
0	7	15	∞
0	7	15	6
0	7	15	3

P

a	b	c	d
-1	0	0	0
-1	a	0	0
-1	a	0	a
-1	a	b	a
-1	a	b	b



Origen: a ; $i = 1$; $n = 4$

Comprobamos que $i < 3$ y D ha sido modificado

Comenzamos la revisión de aristas

Pseudocódigo

$n \leftarrow |V|$; $i \leftarrow 0$; $D \leftarrow [0, \infty, \ldots, \infty]$;
$P \leftarrow [-1, 0, \ldots, 0]$ /* Inicialización de datos /*

while $i < n - 1$ and D modificado en última iteración

for $uv \in E$: /* Para cada arista del grafo /*
$u \leftarrow$ origen de uv; $v \leftarrow$ extremo de uv;
if $D(v) > D(u) + w(uv)$, **then**
$P(v) \leftarrow u$; $D(v) \leftarrow D(u) + w(uv)$
end if

end for

$i \leftarrow i + 1$

End while

Detección de ciclo de peso negativo

Ejemplo (Segunda iteración)

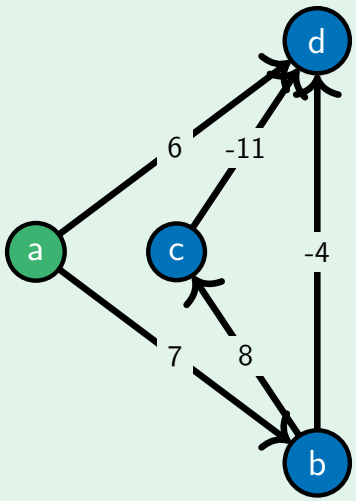
Relajación de aristas

D

a	b	c	d
0	∞	∞	∞
0	7	∞	∞
0	7	15	∞
0	7	15	6
0	7	15	3

P

a	b	c	d
-1	0	0	0
-1	a	0	0
-1	a	0	a
-1	a	b	a
-1	a	b	b



Origen: a ; $i = 1$; $n = 4$

Seleccionamos las aristas ab, ad, bc, bd, cd

Ninguna arista mejora las distancias existentes

No actualizamos distancia ni predecesor de d

Pseudocódigo

$n \leftarrow |V|$; $i \leftarrow 0$; $D \leftarrow [0, \infty, \ldots, \infty]$;
$P \leftarrow [-1, 0, \ldots, 0]$ /* Inicialización de datos /*

while $i < n - 1$ and D modificado en última iteración

for $uv \in E$: /* Para cada arista del grafo /*

$u \leftarrow$ origen de uv; $v \leftarrow$ extremo de uv;

if $D(v) > D(u) + w(uv)$, **then**

$P(v) \leftarrow u$; $D(v) \leftarrow D(u) + w(uv)$

end if

end for

$i \leftarrow i + 1$

End while

Detección de ciclo de peso negativo

Ejemplo (Segunda iteración)

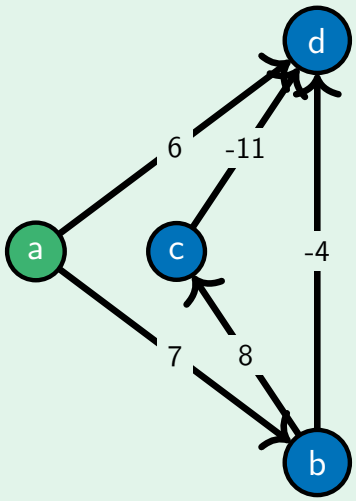
Actualización del contador

D

a	b	c	d
0	∞	∞	∞
0	7	∞	∞
0	7	15	∞
0	7	15	6
0	7	15	3

P

a	b	c	d
-1	0	0	0
-1	a	0	0
-1	a	0	a
-1	a	b	a
-1	a	b	b



Origen: a ; $i = 1 + 1$; $n = 4$

No quedan más aristas

Incrementamos el contador

Pseudocódigo

$n \leftarrow |V|$; $i \leftarrow 0$; $D \leftarrow [0, \infty, \ldots, \infty]$;
$P \leftarrow [-1, 0, \ldots, 0]$ /* Inicialización de datos /*

while $i < n - 1$ and D modificado en última iteración

for $uv \in E$: /* Para cada arista del grafo /*

$u \leftarrow$ origen de uv; $v \leftarrow$ extremo de uv;
if $D(v) > D(u) + w(uv)$, **then**
$P(v) \leftarrow u$; $D(v) \leftarrow D(u) + w(uv)$
end if

end for

$i \leftarrow i + 1$

End while

Detección de ciclo de peso negativo

Ejemplo (Tercera iteración)

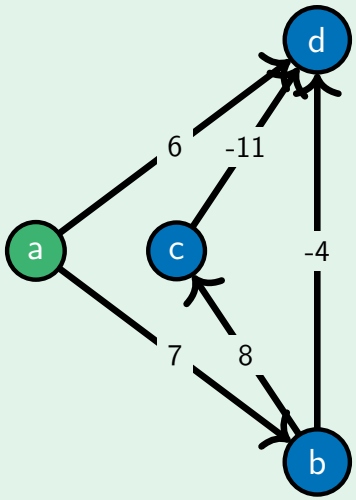
Actualización del contador

D

a	b	c	d
0	∞	∞	∞
0	7	∞	∞
0	7	15	∞
0	7	15	6
0	7	15	3

P

a	b	c	d
-1	0	0	0
-1	a	0	0
-1	a	0	a
-1	a	b	a
-1	a	b	b



Origen: a ; $i = 2$; $n = 4$

Comprobamos que $i < 3$, pero D no ha sido modificado

Comenzamos la detección de ciclos de peso negativo

Pseudocódigo

$n \leftarrow |V|$; $i \leftarrow 0$; $D \leftarrow [0, \infty, \ldots, \infty]$;
$P \leftarrow [-1, 0, \ldots, 0]$ /* Inicialización de datos /*

while $i < n - 1$ and D modificado en última iteración

for $uv \in E$: /* Para cada arista del grafo /*

$u \leftarrow$ origen de uv; $v \leftarrow$ extremo de uv;
if $D(v) > D(u) + w(uv)$, **then**
$P(v) \leftarrow u$; $D(v) \leftarrow D(u) + w(uv)$
end if

end for

$i \leftarrow i + 1$

End while

Detección de ciclo de peso negativo

Algoritmo de Bellmann-Ford

Ejemplo sin ciclos negativos

Ejemplo (Detección de ciclos)

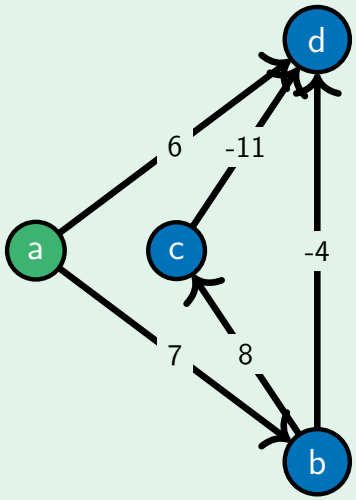
D

a	b	c	d
0	∞	∞	∞
0	7	∞	∞
0	7	15	∞
0	7	15	6
0	7	15	3

P

a	b	c	d
-1	0	0	0
-1	a	0	0
-1	a	0	a
-1	a	b	a
-1	a	b	b

Detección de ciclos



Origen: a ; $i = 2$; $n = 4$

D no ha sido modificado en la última iteración

No hay ciclos de peso negativo

Pseudocódigo

Detección de ciclo de peso negativo

if $i < n - 1$ or D no modificado en la última iteración

Reconstruye *Caminos* utilizando P

/* No hay ciclos negativos /*

End if

else

control $\leftarrow$ correcto

while uv sin comprobar and *control* $\neq$ error

$u \leftarrow$ origen de uv; $v \leftarrow$ extremo de uv;

if $D(v) > D(u) + w(uv)$, **then**

control $\leftarrow$ error

end if

end while

if *control* $=$ **error**

Return "Hay un ciclo de peso negativo"

else Reconstruye *Caminos* utilizando P

end if

Return (*Caminos desde* v_0)

Algoritmo de Bellmann-Ford

Ejemplo sin ciclos negativos

Ejemplo (Detección de ciclos)

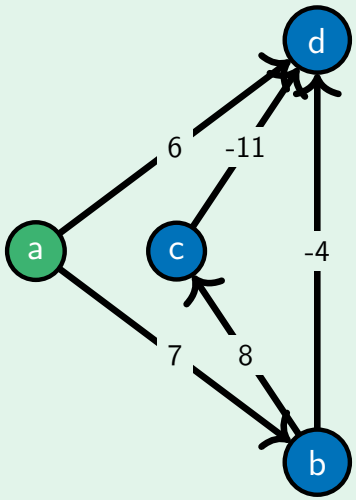
Fin del algoritmo

D

a	b	c	d
0	∞	∞	∞
0	7	∞	∞
0	7	15	∞
0	7	15	6
0	7	15	3

P

a	b	c	d
-1	0	0	0
-1	a	0	0
-1	a	0	a
-1	a	b	a
-1	a	b	b



Origen: a ; $i = 2$; $n = 4$

$P(a, b) = (ab)$, $d(a, b) = 7$

$P(a, c) = (ab, bc)$, $d(a, c) = 15$

$P(a, d) = (ab, bd)$, $d(a, d) = 3$

Pseudocódigo

Detección de ciclo de peso negativo

if $i < n - 1$ or D no modificado en la última iteración

Reconstruye *Caminos* utilizando P

/* No hay ciclos negativos /*

End if

else

control $\leftarrow$ correcto

while *uv* sin comprobar and *control* $\neq$ error

$u \leftarrow$ origen de uv; $v \leftarrow$ extremo de uv;

if $D(v) > D(u) + w(uv)$, **then**

control $\leftarrow$ error

end if

end while

if *control* = **error**

Return "Hay un ciclo de peso negativo"

else Reconstruye *Caminos* utilizando P

end if

Return (*Caminos desde* v_0)

Algoritmo de Bellmann-Ford

Ejemplo con ciclos negativos

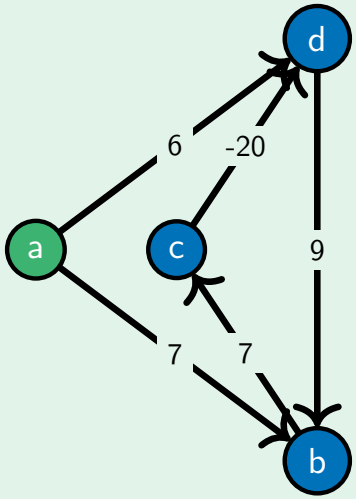
Ejemplo (Inicialización)

D

a	b	c	d
0	∞	∞	∞

P

a	b	c	d
-1	0	0	0



Origen: a ; $i = 0$; $n = 4$

Señalamos el vértice inicial v_0

Iniciamos el contador de iteraciones

Iniciamos los vectores de distancias y predecesores

Pseudocódigo

$n \leftarrow |V|$; $i \leftarrow 0$; $D \leftarrow [0, \infty, \ldots, \infty]$;

$P \leftarrow [-1, 0, \ldots, 0]$ /* Inicialización de datos /*

while $i < n - 1$ and D modificado en última iteración

for $uv \in E$: /* Para cada arista del grafo /*

$u \leftarrow$ origen de uv; $v \leftarrow$ extremo de uv;
if $D(v) > D(u) + w(uv)$, **then**
$P(v) \leftarrow u$; $D(v) \leftarrow D(u) + w(uv)$
end if

end for

$i \leftarrow i + 1$

End while

Detección de ciclo de peso negativo

Ejemplo (Primera iteración)

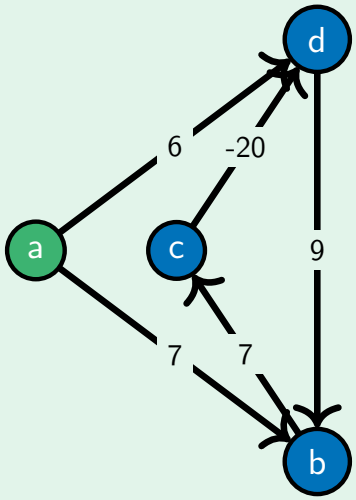
Comprobación del paso

D

a	b	c	d
0	∞	∞	∞

P

a	b	c	d
-1	0	0	0



Origen: a ; $i = 0$; $n = 4$

Comprobamos que $i < 3$ y D ha sido modificado al iniciar

Comenzamos la revisión de aristas

Pseudocódigo

$n \leftarrow |V|$; $i \leftarrow 0$; $D \leftarrow [0, \infty, \ldots, \infty]$;
$P \leftarrow [-1, 0, \ldots, 0]$ /* Inicialización de datos /*

while $i < n - 1$ and D modificado en última iteración

for $uv \in E$: /* Para cada arista del grafo /*
$u \leftarrow$ origen de uv; $v \leftarrow$ extremo de uv;
if $D(v) > D(u) + w(uv)$, **then**
$P(v) \leftarrow u$; $D(v) \leftarrow D(u) + w(uv)$
end if

end for

$i \leftarrow i + 1$

End while

Detección de ciclo de peso negativo

Algoritmo de Bellmann-Ford

Ejemplo sin ciclos negativos

Ejemplo (Primera iteración)

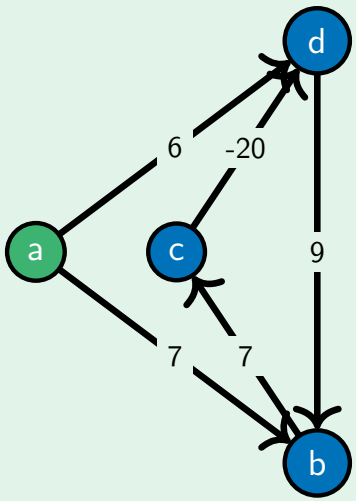
Relajación de aristas

D

a	b	c	d
0	∞	∞	∞
0	7	∞	∞
0	7	∞	6
0	7	14	6
0	7	14	-6
0	3	14	-6

P

a	b	c	d
-1	0	0	0
-1	a	0	0
-1	a	0	a
-1	a	b	a
-1	a	b	c
-1	d	b	c



Origen: a ; $i = 0$; $n = 4$

Recorremos en orden las aristas: ab, ad, bc, cd, db

Todas mejoran la distancia guardada en D

Actualizamos distancias y predecesores con cada una de ellas

Pseudocódigo

$n \leftarrow |V|$; $i \leftarrow 0$; $D \leftarrow [0, \infty, \ldots, \infty]$;
$P \leftarrow [-1, 0, \ldots, 0]$ /* Inicialización de datos /*

while $i < n - 1$ and D modificado en última iteración

for $uv \in E$: /* Para cada arista del grafo /*

$u \leftarrow$ origen de uv; $v \leftarrow$ extremo de uv;

if $D(v) > D(u) + w(uv)$, **then**

$P(v) \leftarrow u$; $D(v) \leftarrow D(u) + w(uv)$

end if

end for

$i \leftarrow i + 1$

End while

Detección de ciclo de peso negativo

Algoritmo de Bellmann-Ford

Ejemplo con ciclos negativos

Ejemplo (Primera iteración)

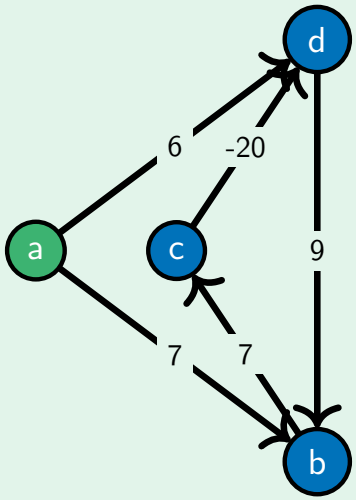
Actualización del contador

D

a	b	c	d
0	∞	∞	∞
0	7	∞	∞
0	7	∞	6
0	7	14	6
0	7	14	-6
0	3	14	-6

P

a	b	c	d
-1	0	0	0
-1	a	0	0
-1	a	0	a
-1	a	b	a
-1	a	b	c
-1	d	b	c



Origen: a ; $i = 0 + 1$; $n = 4$

No quedan más aristas

Incrementamos el contador

Pseudocódigo

$n \leftarrow |V|$; $i \leftarrow 0$; $D \leftarrow [0, \infty, \ldots, \infty]$;
$P \leftarrow [-1, 0, \ldots, 0]$ /* Inicialización de datos /*

while $i < n - 1$ and D modificado en última iteración

for $uv \in E$: /* Para cada arista del grafo /*

$u \leftarrow$ origen de uv; $v \leftarrow$ extremo de uv;
if $D(v) > D(u) + w(uv)$, **then**
$P(v) \leftarrow u$; $D(v) \leftarrow D(u) + w(uv)$
end if

end for

$i \leftarrow i + 1$

End while

Detección de ciclo de peso negativo

Ejemplo (Segunda iteración)

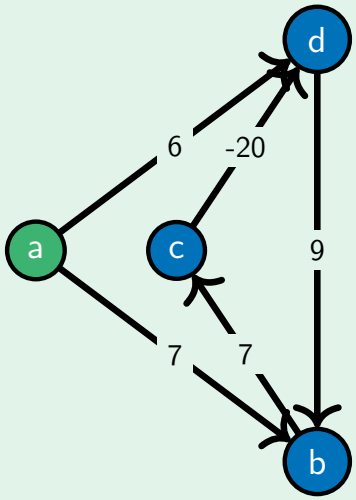
Comprobación del paso

D

a	b	c	d
...	...	...	...
0	3	14	-6

P

a	b	c	d
...	...	...	...
-1	d	b	c



Origen: a ; $i = 1$; $n = 4$

Comprobamos que $i < 3$ y D ha sido modificado en el paso anterior

Comenzamos la revisión de aristas

Pseudocódigo

$n \leftarrow |V|$; $i \leftarrow 0$; $D \leftarrow [0, \infty, \ldots, \infty]$;
$P \leftarrow [-1, 0, \ldots, 0]$ /* Inicialización de datos /*

while $i < n - 1$ and D modificado en última iteración

for $uv \in E$: /* Para cada arista del grafo /*
$u \leftarrow$ origen de uv; $v \leftarrow$ extremo de uv;
if $D(v) > D(u) + w(uv)$, **then**
$P(v) \leftarrow u$; $D(v) \leftarrow D(u) + w(uv)$
end if

end for

$i \leftarrow i + 1$

End while

Detección de ciclo de peso negativo

Ejemplo (Segunda iteración)

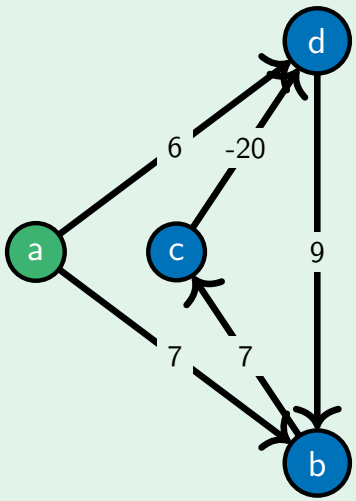
Relajación de aristas

D

a	b	c	d
...	...	...	...
0	3	14	-6
0	3	10	-6
0	3	10	-10
0	-1	10	-10

P

a	b	c	d
...	...	...	...
-1	d	b	c
-1	d	b	c
-1	d	b	c
-1	d	b	c



Origen: a ; $i = 1$; $n = 4$

Recorremos en orden las aristas ab, ad, bc, cd, db

Solo mejoran las distancias: bc, cd, db

Actualizamos distancia y predecesores en esos casos

Pseudocódigo

$n \leftarrow |V|$; $i \leftarrow 0$; $D \leftarrow [0, \infty, \ldots, \infty]$;
$P \leftarrow [-1, 0, \ldots, 0]$ /* Inicialización de datos /*

while $i < n - 1$ and D modificado en última iteración

for $uv \in E$: /* Para cada arista del grafo /*

$u \leftarrow$ origen de uv; $v \leftarrow$ extremo de uv;

if $D(v) > D(u) + w(uv)$, **then**

$P(v) \leftarrow u$; $D(v) \leftarrow D(u) + w(uv)$

end if

end for

$i \leftarrow i + 1$

End while

Detección de ciclo de peso negativo

Ejemplo (Segunda iteración)

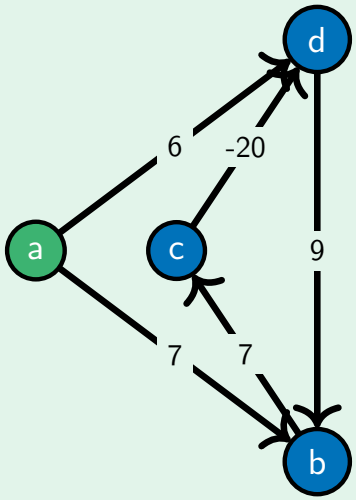
Actualización del contador

D

a	b	c	d
...	...	...	...
0	3	14	-6
0	3	10	-6
0	3	10	-10
0	-1	10	-10

P

a	b	c	d
...	...	...	...
-1	d	b	c
-1	d	b	c
-1	d	b	c
-1	d	b	c



Origen: a ; $i = 1 + 1$; $n = 4$

No quedan más aristas

Incrementamos el contador

Pseudocódigo

$n \leftarrow |V|$; $i \leftarrow 0$; $D \leftarrow [0, \infty, \ldots, \infty]$;
$P \leftarrow [-1, 0, \ldots, 0]$ /* Inicialización de datos /*

while $i < n - 1$ and D modificado en última iteración

for $uv \in E$: /* Para cada arista del grafo /*

$u \leftarrow$ origen de uv; $v \leftarrow$ extremo de uv;
if $D(v) > D(u) + w(uv)$, **then**
$P(v) \leftarrow u$; $D(v) \leftarrow D(u) + w(uv)$
end if

end for

$i \leftarrow i + 1$

End while

Detección de ciclo de peso negativo

Algoritmo de Bellmann-Ford

Ejemplo con ciclos negativos

Ejemplo (Tercera iteración)

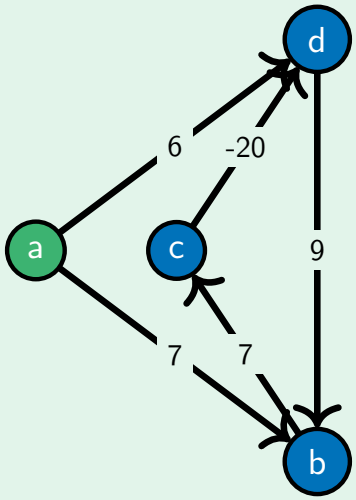
Comprobación del paso

D

a	b	c	d
...	...	...	...
0	-1	10	-10

P

a	b	c	d
...	...	...	...
-1	d	b	c



Origen: a ; $i = 2$; $n = 4$

Comprobamos que $i < 3$ y D ha sido modificado en el paso anterior

Comenzamos la revisión de aristas

Pseudocódigo

$n \leftarrow |V|$; $i \leftarrow 0$; $D \leftarrow [0, \infty, \ldots, \infty]$;
$P \leftarrow [-1, 0, \ldots, 0]$ /* Inicialización de datos /*

while $i < n - 1$ and D modificado en última iteración

for $uv \in E$: /* Para cada arista del grafo /*
$u \leftarrow$ origen de uv; $v \leftarrow$ extremo de uv;
if $D(v) > D(u) + w(uv)$, **then**
$P(v) \leftarrow u$; $D(v) \leftarrow D(u) + w(uv)$
end if

end for

$i \leftarrow i + 1$

End while

Detección de ciclo de peso negativo

Algoritmo de Bellmann-Ford

Ejemplo con ciclos negativos

Ejemplo (Tercera iteración)

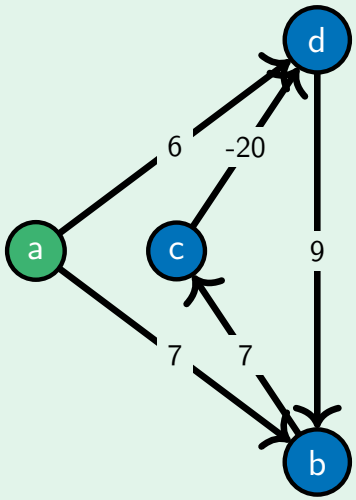
Relajación de aristas

D

a	b	c	d
...	...	...	...
0	-1	10	-10
0	-1	6	-10
0	-1	6	-14
0	-5	6	-14

P

a	b	c	d
...	...	...	...
-1	d	b	c
-1	d	b	c
-1	d	b	c
-1	d	b	c



Origen: a ; $i = 2$; $n = 4$

Recorremos en orden las aristas ab, ad, bc, cd, db

Solo mejoran las distancias: bc, cd, db

Actualizamos distancia y predecesores en esos pasos

Pseudocódigo

$n \leftarrow |V|$; $i \leftarrow 0$; $D \leftarrow [0, \infty, \ldots, \infty]$;
$P \leftarrow [-1, 0, \ldots, 0]$ /* Inicialización de datos /*

while $i < n - 1$ and D modificado en última iteración

for $uv \in E$: /* Para cada arista del grafo /*

$u \leftarrow$ origen de uv; $v \leftarrow$ extremo de uv;

if $D(v) > D(u) + w(uv)$, **then**

$P(v) \leftarrow u$; $D(v) \leftarrow D(u) + w(uv)$

end if

end for

$i \leftarrow i + 1$

End while

Detección de ciclo de peso negativo

Algoritmo de Bellmann-Ford

Ejemplo con ciclos negativos

Ejemplo (Tercera iteración)

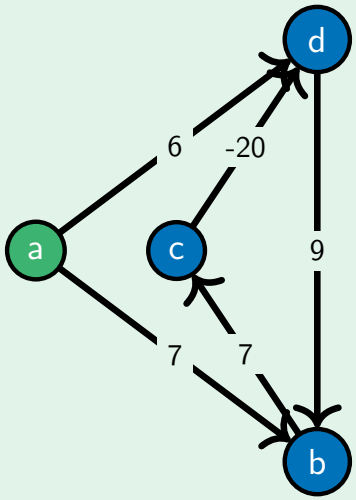
Actualización del contador

D

a	b	c	d
...	...	...	...
0	-1	10	-10
0	-1	6	-10
0	-1	6	-14
0	-5	6	-14

P

a	b	c	d
...	...	...	...
-1	d	b	c
-1	d	b	c
-1	d	b	c
-1	d	b	c



Origen: a ; $i = 2 + 1$; $n = 4$

No quedan más aristas

Incrementamos el contador

Pseudocódigo

$n \leftarrow |V|$; $i \leftarrow 0$; $D \leftarrow [0, \infty, \ldots, \infty]$;
$P \leftarrow [-1, 0, \ldots, 0]$ /* Inicialización de datos /*

while $i < n - 1$ and D modificado en última iteración

for $uv \in E$: /* Para cada arista del grafo /*

$u \leftarrow$ origen de uv; $v \leftarrow$ extremo de uv;
if $D(v) > D(u) + w(uv)$, **then**
$P(v) \leftarrow u$; $D(v) \leftarrow D(u) + w(uv)$
end if

end for

$i \leftarrow i + 1$

End while

Detección de ciclo de peso negativo

Ejemplo (Cuarta iteración)

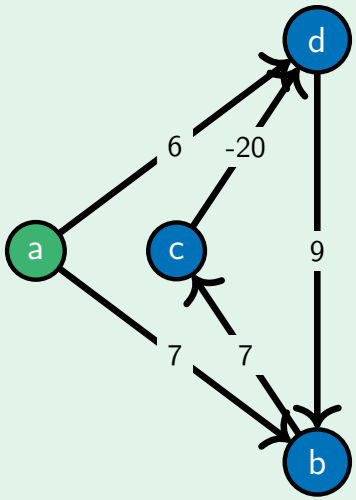
Comprobación del paso

D

a	b	c	d
...	...	...	...
0	-5	6	-14

P

a	b	c	d
...	...	...	...
-1	d	b	c



Origen: a ; $i = 2$; $n = 4$

Comprobamos que $i = 3$ y D ha sido modificado en el último paso

Comenzamos la detección de ciclos de peso negativo

Pseudocódigo

$n \leftarrow |V|$; $i \leftarrow 0$; $D \leftarrow [0, \infty, \ldots, \infty]$;
$P \leftarrow [-1, 0, \ldots, 0]$ /* Inicialización de datos /*

while $i < n - 1$ and D modificado en última iteración

for $uv \in E$: /* Para cada arista del grafo /*

$u \leftarrow$ origen de uv; $v \leftarrow$ extremo de uv;
if $D(v) > D(u) + w(uv)$, **then**
$P(v) \leftarrow u$; $D(v) \leftarrow D(u) + w(uv)$
end if

end for

$i \leftarrow i + 1$

End while

Detección de ciclo de peso negativo

Algoritmo de Bellmann-Ford

Ejemplo con ciclos negativos

Ejemplo (Detección de ciclos)

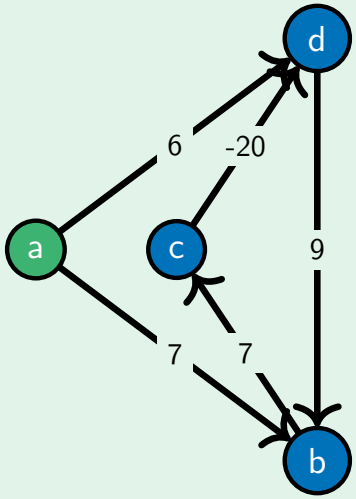
Detección de ciclos

D

a	b	c	d
...	...	...	...
0	-5	6	-14

P

a	b	c	d
...	...	...	...
-1	d	b	c



Origen: a ; $i = 2$; $n = 4$

D ha sido modificado en la última iteración

Comprobamos las aristas para actualizar el control

ab: $D(b) = -5 < 7 = D(a) + w(ab)$

ad: $D(d) = -14 < 6 = D(a) + w(ad)$

bc: $D(c) = 6 > -1 = D(b) + w(bc) \implies control = \text{error}$

Pseudocódigo

Detección de ciclo de peso negativo

if $i < n - 1$ or D no modificado en la última iteración

Reconstruye *Caminos* utilizando P

/* No hay ciclos negativos /*

End if

else

control $\leftarrow$ correcto

while uv sin comprobar and *control* $\neq$ error

$u \leftarrow$ origen de uv; $v \leftarrow$ extremo de uv;

if $D(v) > D(u) + w(uv)$, **then**

control $\leftarrow$ error

end if

end while

if *control* = error

Return "Hay un ciclo de peso negativo"

else Reconstruye *Caminos* utilizando P

end if

Return (*Caminos desde* v_0)

Algoritmo de Bellmann-Ford

Ejemplo con ciclos negativos

Ejemplo (Detección de ciclos)

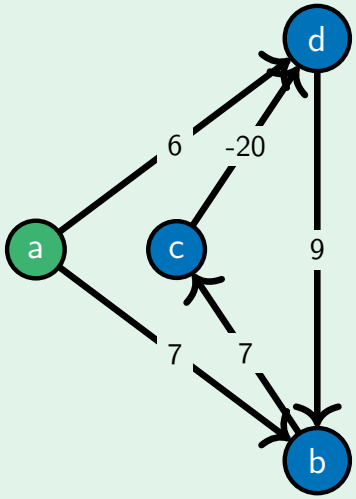
Detección de ciclos

D

a	b	c	d
...	...	...	...
0	-5	6	-14

P

a	b	c	d
...	...	...	...
-1	d	b	c



Origen: a ; $i = 2$; $n = 4$

El algoritmo detecta un ciclo de peso negativo

$b \mapsto c \mapsto d \mapsto b$ con peso $7 + (-20) + 9 = -4$

Pseudocódigo

Detección de ciclo de peso negativo

if $i < n - 1$ or D no modificado en la última iteración

Reconstruye *Caminos* utilizando P

/* No hay ciclos negativos /*

End if

else

control $\leftarrow$ correcto

while uv sin comprobar and *control* $\neq$ error

$u \leftarrow$ origen de uv; $v \leftarrow$ extremo de uv;

if $D(v) > D(u) + w(uv)$, **then**

control $\leftarrow$ error

end if

end while

if *control* = error

Return "Hay un ciclo de peso negativo"

else Reconstruye *Caminos* utilizando P

end if

Return (*Caminos desde* v_0)

Algoritmo de Floyd-Warshall

Estructura del algoritmo

Input: Grafo (o digrafo) simple con pesos $G = (V, E, w)$.

Clave: Construir una sucesión de matrices $W^0, W^1, \ldots, W^n$ donde el elemento $W^k(i,j)$ de la matriz W^k indique la longitud del camino mínimo de v_i a v_j utilizando vértices interiores del conjunto $\{v_1, v_2, \ldots, v_k\}$.

Inicialización: $W^0 = W$ matriz de pesos, P^0 matriz de predecesores.

Iteración: Se construye la matriz W^k a partir de W^{k-1}:

- Si la distancia anterior $W^{k-1}(i,j)$ de V_i a v_j puede ser mejorada por un camino que pasa por v_k, actualizamos: $W^k(i,j) = W^{k-1}(i,k) + W^{k-1}(k,j)$
- En otro caso, mantenemos $W^k(i,j) = W^{k-1}(i,j)$

Output: Distancia y caminos mínimos entre cada par de vértices v_i, v_j.

Observaciones

- Si hay un ciclo negativo, algún elemento de la diagonal principal se hará negativo en alguna de las matrices W^k.
- Para almacenar los caminos mínimos, se construye una sucesión de matrices de predecesores $P^0, P^1, \ldots, P^n$, del siguiente modo:
  - Al iniciar los datos, si existe la arista ij, se tiene $P^0_{ij} = i$. Si no hay tal arista, entonces se deja en blanco.
  - En el paso k, si no se ha producido cambio en $W^k(i,j)$, entonces $P^k_{ij} = P^{k-1}_{ij}$. Si se ha producido cambio, entonces $P^k_{ij} = P^{k-1}_{kj}$.
- Al final, el elemento P^n_{ij} indica el último vértice antes de v_j en el camino de longitud mínima que va desde v_i hasta v_j. Si el elemento está en blanco, significa que no hay ningún camino que los una.

Pseudocódigo: Inicio y actualización de pesos y predecesores

$n \leftarrow |V|$; $W^0 \leftarrow W$ (matriz de pesos); $P(i,j) \leftarrow i$ (si ij es arista), $-$ (otro caso);

/* Inicialización de datos /*

for k from 1 to n: /* Iteración para cada matriz /*

  for i from 1 to n: /* Recorrido de las entradas de la matriz /*

    for j from 1 to n: /* Recorrido de las entradas de la matriz /*

      if $W^{k-1}(i,j) > W^{k-1}(i,k) + W^{k-1}(k,j)$, **then**

        $W^k(i,j) \leftarrow W^{k-1}(i,k) + W^{k-1}(k,j)$

        $P(i,j) \leftarrow P(k,j)$

      else

        $W^k(i,j) \leftarrow W^{k-1}(i,j)$

      end if

    end for

  end for

End for

Detección de ciclo de peso negativo /* Continúa en la siguiente página /*

Pseudocódigo: Detección de ciclos negativos y recuperación de caminos

Detección de ciclo de peso negativo

for i from 1 to n:

    if $W^n(i,i) < 0$, **then**

        Return "Hay un ciclo de peso negativo"

    end if

end for

Return (W^n, P)

Recuperación de caminos

$Camino(i,j) \leftarrow [Camino(i, P(i,j)), j]$

donde $Camino(i,i) = i$

Return (*Caminos desde* v_i *hasta* v_j)

Algoritmo de Floyd-Warshall

Ejemplo

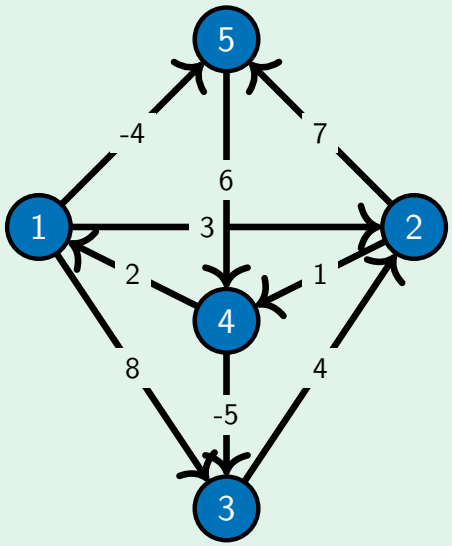
Ejemplo (Inicialización)

W^0

0	3	8	∞	-4
∞	0	∞	1	7
∞	4	0	∞	∞
2	∞	-5	0	∞
∞	∞	∞	6	0

P^0

-	1	1	-	1
-	-	-	2	2
-	3	-	-	-
4	-	4	-	-
-	-	-	5	-



Iniciamos la matriz de pesos W^0

Iniciamos la matriz de predecesores P

Pseudocódigo

$n \leftarrow |V|$; $W^0 \leftarrow W$ (matriz de pesos);

$P(i,j) \leftarrow i$ (si ij es arista), $-$ (otro caso);

/* Inicialización de datos /*

for k from 1 to n:

  for i from 1 to n:

    for j from 1 to n:

      if $W^{k-1}(i,j) > W^{k-1}(i,k) + W^{k-1}(k,j)$:

        $W^k(i,j) \leftarrow W^{k-1}(i,k) + W^{k-1}(k,j)$

        $P(i,j) \leftarrow P(k,j)$

      else

        $W^k(i,j) \leftarrow W^{k-1}(i,j)$

      end if

    end for

  end for

end for

Detección de ciclo de peso negativo

Algoritmo de Floyd-Warshall

Ejemplo

Ejemplo (Primera iteración)

W^0

0	3	8	∞	-4
∞	0	∞	1	7
∞	4	0	∞	∞
2	∞	-5	0	∞
∞	∞	∞	6	0

P^0

-	1	1	-	1
-	-	-	2	2
-	3	-	-	-
4	-	4	-	-
-	-	-	5	-

W^1

0	3	8	∞	-4
∞	0	∞	1	7
∞	4	0	∞	∞
2	5	-5	0	-2
∞	∞	∞	6	0

P^1

-	1	1	-	1
-	-	-	2	2
-	3	-	-	-
4	1	4	-	1
-	-	-	5	-

Para $k = 1$, $i = 1, 2, 3, 5$, $j = 1, \ldots, 5$, no hay cambios

Para $k = 1$, $i = 4$, $j = 1, 3, 4$, no hay cambios

Hay cambios para $k = 1$, $i = 4$, $j = 2, 5$;

Pseudocódigo

$n \leftarrow |V|$; $W^0 \leftarrow W$ (matriz de pesos);

$P(i,j) \leftarrow i$ (si ij es arista), $-$ (otro caso);

/* Inicialización de datos /*

for k from 1 to n:

  for i from 1 to n:

    for j from 1 to n:

      if $W^{k-1}(i,j) > W^{k-1}(i,k) + W^{k-1}(k,j)$:

        $W^k(i,j) \leftarrow W^{k-1}(i,k) + W^{k-1}(k,j)$

        $P(i,j) \leftarrow P(k,j)$

      else

        $W^k(i,j) \leftarrow W^{k-1}(i,j)$

      end if

    end for

  end for

end for

Detección de ciclo de peso negativo

Ejemplo (Segunda iteración)

W^1

0	3	8	∞	-4
∞	0	∞	1	7
∞	4	0	∞	∞
2	5	-5	0	-2
∞	∞	∞	6	0

W^2

0	3	8	4	-4
∞	0	∞	1	7
∞	4	0	∞	∞
2	5	-5	0	-2
∞	∞	∞	6	0

P^1

-	1	1	-	1
-	-	-	2	2
-	3	-	-	-
4	1	4	-	1
-	-	-	5	-

P^2

-	1	1	2	1
-	-	-	2	2
-	3	-	-	-
4	1	4	-	1
-	-	-	5	-

Para $k = 2$, $i = 1$, $j = 1, 2, 3, 5$, no hay cambios

Hay cambios para $k = 2$, $i = 1$, $j = 4$;

Para $k = 2$, $i = 2$, $j = 1, \ldots, 5$, no hay cambios

Pseudocódigo

$n \leftarrow |V|$; $W^0 \leftarrow W$ (matriz de pesos);

$P(i, j) \leftarrow i$ (si ij es arista), $-$ (otro caso);

/* Inicialización de datos /*

for k from 1 to n:

  for i from 1 to n:

    for j from 1 to n:

      if $W^{k-1}(i, j) > W^{k-1}(i, k) + W^{k-1}(k, j)$:

        $W^k(i, j) \leftarrow W^{k-1}(i, k) + W^{k-1}(k, j)$

        $P(i, j) \leftarrow P(k, j)$

      else

        $W^k(i, j) \leftarrow W^{k-1}(i, j)$

      end if

    end for

  end for

end for

Detección de ciclo de peso negativo

Ejemplo (Segunda iteración)

W^1

0	3	8	∞	-4
∞	0	∞	1	7
∞	4	0	∞	∞
2	5	-5	0	-2
∞	∞	∞	6	0

P^1

-	1	1	-	1
-	-	-	2	2
-	3	-	-	-
4	1	4	-	1
-	-	-	5	-

W^2

0	3	8	4	-4
∞	0	∞	1	7
∞	4	0	5	11
2	5	-5	0	-2
∞	∞	∞	6	0

P^2

-	1	1	2	1
-	-	-	2	2
-	3	-	2	2
4	1	4	-	1
-	-	-	5	-

Para $k = 2$, $i = 3$, $j = 1, 2, 3$, no hay cambios

Hay cambios para $k = 2$, $i = 3$, $j = 4, 5$

Para $k = 2$, $i = 4, 5$, $j = 1, \ldots, 5$ no hay cambios

Pseudocódigo

$n \leftarrow |V|$; $W^0 \leftarrow W$ (matriz de pesos);

$P(i,j) \leftarrow i$ (si ij es arista), $-$ (otro caso);

/* Inicialización de datos /*

for k from 1 to n:

  for i from 1 to n:

    for j from 1 to n:

      if $W^{k-1}(i,j) > W^{k-1}(i,k) + W^{k-1}(k,j)$:

        $W^k(i,j) \leftarrow W^{k-1}(i,k) + W^{k-1}(k,j)$

        $P(i,j) \leftarrow P(k,j)$

      else

        $W^k(i,j) \leftarrow W^{k-1}(i,j)$

      end if

    end for

  end for

end for

Detección de ciclo de peso negativo

Ejemplo (Tercera iteración)

W^2

0	3	8	4	-4
∞	0	∞	1	7
∞	4	0	5	11
2	5	-5	0	-2
∞	∞	∞	6	0

P^2

-	1	1	2	1
-	-	-	2	2
-	3	-	2	2
4	1	4	-	1
-	-	-	5	-

W^3

0	3	8	4	-4
∞	0	∞	1	7
∞	4	0	5	11
2	-1	-5	0	-2
∞	∞	∞	6	0

P^3

-	1	1	2	1
-	-	-	2	2
-	3	-	2	2
4	3	4	-	1
-	-	-	5	-

Para $k = 3$, $i = 1, 2, 3, 5$, $j = 1, \ldots, 5$, no hay cambios

Para $k = 3$, $i = 4$, $j = 1, 3, 4, 5$, no hay cambios

Hay cambios para $k = 3$, $i = 4$, $j = 2$

Pseudocódigo

$n \leftarrow |V|$; $W^0 \leftarrow W$ (matriz de pesos);

$P(i,j) \leftarrow i$ (si ij es arista), $-$ (otro caso);

/* Inicialización de datos /*

for k from 1 to n:

  for i from 1 to n:

    for j from 1 to n:

      if $W^{k-1}(i,j) > W^{k-1}(i,k) + W^{k-1}(k,j)$:

        $W^k(i,j) \leftarrow W^{k-1}(i,k) + W^{k-1}(k,j)$

        $P(i,j) \leftarrow P(k,j)$

      else

        $W^k(i,j) \leftarrow W^{k-1}(i,j)$

      end if

    end for

  end for

end for

Detección de ciclo de peso negativo

Algoritmo de Floyd-Warshall

Ejemplo

Ejemplo (Cuarta iteración)

W^3

0	3	8	4	-4
∞	0	∞	1	7
∞	4	0	5	11
2	-1	-5	0	-2
∞	∞	∞	6	0

W^4

0	3	1	4	-4
∞	0	∞	1	7
∞	4	0	5	11
2	-1	-5	0	-2
∞	∞	∞	6	0

P^3

-	1	1	2	1
-	-	-	2	2
-	3	-	2	2
4	3	4	-	1
-	-	-	5	-

P^4

-	1	4	2	1
-	-	-	2	2
-	3	-	2	2
4	3	4	-	1
-	-	-	5	-

Para $k = 4$, $i = 1$, $j = 1, 2, 4, 5$, no hay cambios

Hay cambios para $k = 4$, $i = 1$, $j = 3$

Pseudocódigo

$n \leftarrow |V|$; $W^0 \leftarrow W$ (matriz de pesos);

$P(i,j) \leftarrow i$ (si ij es arista), $-$ (otro caso);

/* Inicialización de datos /*

for k from 1 to n:

  for i from 1 to n:

    for j from 1 to n:

      if $W^{k-1}(i,j) > W^{k-1}(i,k) + W^{k-1}(k,j)$:

        $W^k(i,j) \leftarrow W^{k-1}(i,k) + W^{k-1}(k,j)$

        $P(i,j) \leftarrow P(k,j)$

      else

        $W^k(i,j) \leftarrow W^{k-1}(i,j)$

      end if

    end for

  end for

end for

Detección de ciclo de peso negativo

Ejemplo (Cuarta iteración)

W^3

0	3	8	4	-4
∞	0	∞	1	7
∞	4	0	5	11
2	-1	-5	0	-2
∞	∞	∞	6	0

W^4

0	3	1	4	-4
3	0	-4	1	-1
∞	4	0	5	11
2	-1	-5	0	-2
∞	∞	∞	6	0

P^3

-	1	1	2	1
-	-	-	2	2
-	3	-	2	2
4	3	4	-	1
-	-	-	5	-

P^4

-	1	4	2	1
4	-	4	2	1
-	3	-	2	2
4	3	4	-	1
-	-	-	5	-

Para $k = 4$, $i = 2$, $j = 2, 4, 5$, no hay cambios

Hay cambios para $k = 4$, $i = 2$, $j = 1, 3, 5$

Pseudocódigo

$n \leftarrow |V|$; $W^0 \leftarrow W$ (matriz de pesos);

$P(i,j) \leftarrow i$ (si ij es arista), $-$ (otro caso);

/* Inicialización de datos /*

for k from 1 to n:

  for i from 1 to n:

    for j from 1 to n:

      if $W^{k-1}(i,j) > W^{k-1}(i,k) + W^{k-1}(k,j)$:

        $W^k(i,j) \leftarrow W^{k-1}(i,k) + W^{k-1}(k,j)$

        $P(i,j) \leftarrow P(k,j)$

      else

        $W^k(i,j) \leftarrow W^{k-1}(i,j)$

      end if

    end for

  end for

end for

Detección de ciclo de peso negativo

Algoritmo de Floyd-Warshall

Ejemplo

Ejemplo (Cuarta iteración)

W^3

0	3	8	4	-4
∞	0	∞	1	7
∞	4	0	5	11
2	-1	-5	0	-2
∞	∞	∞	6	0

P^3

-	1	1	2	1
-	-	-	2	2
-	3	-	2	2
4	3	4	-	1
-	-	-	5	-

W^4

0	3	1	4	-4
3	0	-4	1	-1
7	4	0	5	3
2	-1	-5	0	-2
∞	∞	∞	6	0

P^4

-	1	4	2	1
4	-	4	2	1
4	3	-	2	1
4	3	4	-	1
-	-	-	5	-

Para $k = 4$, $i = 3$, $j = 2, 3, 4$, no hay cambios

Hay cambios para $k = 4$, $i = 3$, $j = 1, 5$

Para $k = 4$, $i = 4$, $j = 1, \ldots, 5$, no hay cambios

Pseudocódigo

$n \leftarrow |V|$; $W^0 \leftarrow W$ (matriz de pesos);

$P(i,j) \leftarrow i$ (si ij es arista), $-$ (otro caso);

/* Inicialización de datos /*

for k from 1 to n:

  for i from 1 to n:

    for j from 1 to n:

      if $W^{k-1}(i,j) > W^{k-1}(i,k) + W^{k-1}(k,j)$:

        $W^k(i,j) \leftarrow W^{k-1}(i,k) + W^{k-1}(k,j)$

        $P(i,j) \leftarrow P(k,j)$

      else

        $W^k(i,j) \leftarrow W^{k-1}(i,j)$

      end if

    end for

  end for

end for

Detección de ciclo de peso negativo

Algoritmo de Floyd-Warshall

Ejemplo

Ejemplo (Cuarta iteración)

W^3

0	3	8	4	-4
∞	0	∞	1	7
∞	4	0	5	11
2	-1	-5	0	-2
∞	∞	∞	6	0

W^4

0	3	1	4	-4
3	0	-4	1	-1
7	4	0	5	3
2	-1	-5	0	-2
8	5	1	6	0

P^3

-	1	1	2	1
-	-	-	2	2
-	3	-	2	2
4	3	4	-	1
-	-	-	5	-

P^4

-	1	4	2	1
4	-	4	2	1
4	3	-	2	1
4	3	4	-	1
4	3	4	5	-

Hay cambios para $k = 4$, $i = 5$, $j = 1, 2, 3$

Para $k = 4$, $i = 5$, $j = 4, 5$, no hay cambios

Pseudocódigo

$n \leftarrow |V|$; $W^0 \leftarrow W$ (matriz de pesos);

$P(i,j) \leftarrow i$ (si ij es arista), $-$ (otro caso);

/* Inicialización de datos /*

for k from 1 to n:

  for i from 1 to n:

    for j from 1 to n:

      if $W^{k-1}(i,j) > W^{k-1}(i,k) + W^{k-1}(k,j)$:

        $W^k(i,j) \leftarrow W^{k-1}(i,k) + W^{k-1}(k,j)$

        $P(i,j) \leftarrow P(k,j)$

      else

        $W^k(i,j) \leftarrow W^{k-1}(i,j)$

      end if

    end for

  end for

end for

Detección de ciclo de peso negativo

Algoritmo de Floyd-Warshall

Ejemplo

Ejemplo (Quinta iteración)

W^4

0	3	1	4	-4
3	0	-4	1	-1
7	4	0	5	3
2	-1	-5	0	-2
8	5	1	6	0

P^4

-	1	4	2	1
4	-	4	2	1
4	3	-	2	1
4	3	4	-	1
4	3	4	5	-

W^5

0	1	-3	2	-4
3	0	-4	1	-1
7	4	0	5	3
2	-1	-5	0	-2
8	5	1	6	0

P^5

-	3	4	5	1
4	-	4	2	1
4	3	-	2	1
4	3	4	-	1
4	3	4	5	-

Para $k = 5$, $i = 1$, $j = 1, 5$, no hay cambios

Hay cambios para $k = 5$, $i = 5$, $j = 2, 3, 4$

Para $k = 5$, $i = 2, 3, 4, 5$, $j = 1, \ldots, 5$, no hay cambios

Pseudocódigo

$n \leftarrow |V|$; $W^0 \leftarrow W$ (matriz de pesos);

$P(i, j) \leftarrow i$ (si ij es arista), $-$ (otro caso);

/* Inicialización de datos /*

for k from 1 to n:

  for i from 1 to n:

    for j from 1 to n:

      if $W^{k-1}(i, j) > W^{k-1}(i, k) + W^{k-1}(k, j)$:

        $W^k(i, j) \leftarrow W^{k-1}(i, k) + W^{k-1}(k, j)$

        $P(i, j) \leftarrow P(k, j)$

      else

        $W^k(i, j) \leftarrow W^{k-1}(i, j)$

      end if

    end for

  end for

end for

Detección de ciclo de peso negativo

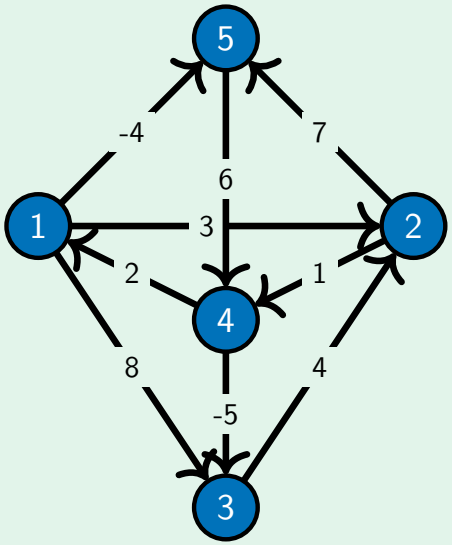
Ejemplo (Fin de las iteraciones)

W^5

0	1	-3	2	-4
3	0	-4	1	-1
7	4	0	5	3
2	-1	-5	0	-2
8	5	1	6	0

P^5

-	3	4	5	1
4	-	4	2	1
4	3	-	2	1
4	3	4	-	1
4	3	4	5	-



Hemos completado las iteraciones en k

Pasamos a la detección de ciclos

Pseudocódigo

$n \leftarrow |V|$; $W^0 \leftarrow W$ (matriz de pesos);

$P(i,j) \leftarrow i$ (si ij es arista), $-$ (otro caso);

/* Inicialización de datos /*

for k from 1 to n:

  for i from 1 to n:

    for j from 1 to n:

      if $W^{k-1}(i,j) > W^{k-1}(i,k) + W^{k-1}(k,j)$:

        $W^k(i,j) \leftarrow W^{k-1}(i,k) + W^{k-1}(k,j)$

        $P(i,j) \leftarrow P(k,j)$

      else

        $W^k(i,j) \leftarrow W^{k-1}(i,j)$

      end if

    end for

  end for

end for

Detección de ciclo de peso negativo

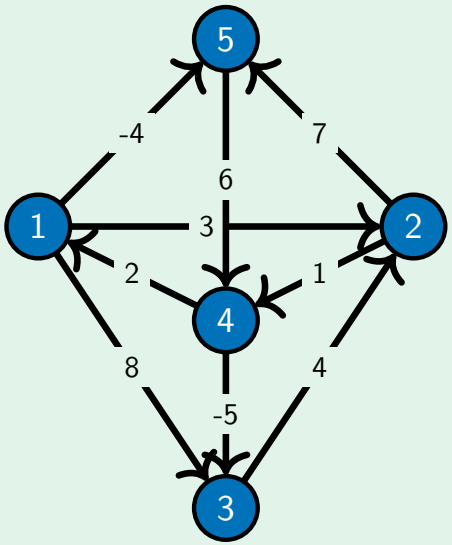
Ejemplo (Detección de ciclos)

W^5

0	1	-3	2	-4
3	0	-4	1	-1
7	4	0	5	3
2	-1	-5	0	-2
8	5	1	6	0

P^5

-	3	4	5	1
4	-	4	2	1
4	3	-	2	1
4	3	4	-	1
4	3	4	5	-



No tenemos ciclos negativos pues $W^5(i,i) \geq 0$ para $i = 1, \ldots, 5$

Pasamos a la recuperación de caminos

Pseudocódigo

Detección de ciclo de peso negativo

for i from 1 to n:

  if $W^n(i,i) < 0$, **then**

    Return "Hay un ciclo de peso negativo"

  end if

end for

Return (W^n, P)

Recuperación de caminos

$Camino(i,j) \leftarrow [Camino(i, P(i,j)), j]$

donde $Camino(i,i) = i$

Return (*Caminos desde* v_i *hasta* v_j)

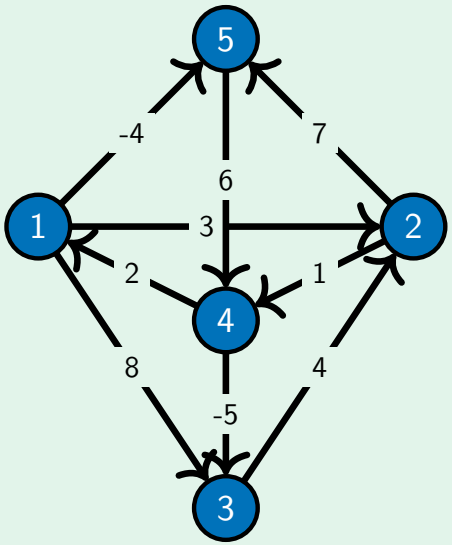
Ejemplo (Recuperación de caminos)

W^5

0	1	-3	2	-4
3	0	-4	1	-1
7	4	0	5	3
2	-1	-5	0	-2
8	5	1	6	0

P^5

-	3	4	5	1
4	-	4	2	1
4	3	-	2	1
4	3	4	-	1
4	3	4	5	-



Mostramos algunas distancias y caminos recuperados

$d(1,4) = 2, \quad 1 \mapsto 5 \mapsto 4$

$d(2,5) = -1, \quad 2 \mapsto 4 \mapsto 1 \mapsto 5$

$d(4,3) = -5, \quad 4 \mapsto 3$

Pseudocódigo

Detección de ciclo de peso negativo

for i from 1 to n:

  if $W^n(i,i) < 0$, **then**

    Return "Hay un ciclo de peso negativo"

  end if

end for

Return (W^n, P)

Recuperación de caminos

$Camino(i,j) \leftarrow [Camino(i, P(i,j)), j]$

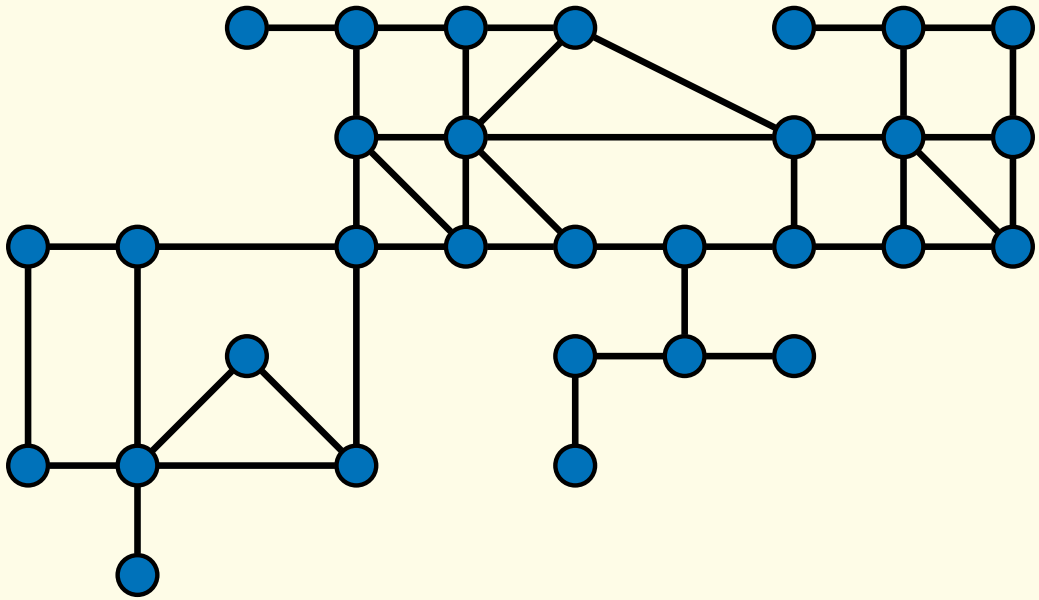
donde $Camino(i,i) = i$

Return (*Caminos desde* v_i *hasta* v_j)

Problemas de ubicación de servicios. Excentricidad, centro y diámetro.

Motivación

Supongamos que el siguiente grafo representa una ciudad, donde los vértices son bloques de edificios y las aristas son calles que los comunican.



¿Dónde debemos ubicar determinados servicios (supermercados, bomberos, policía, restaurantes...) para que la población de la ciudad esté "bien atendida"?

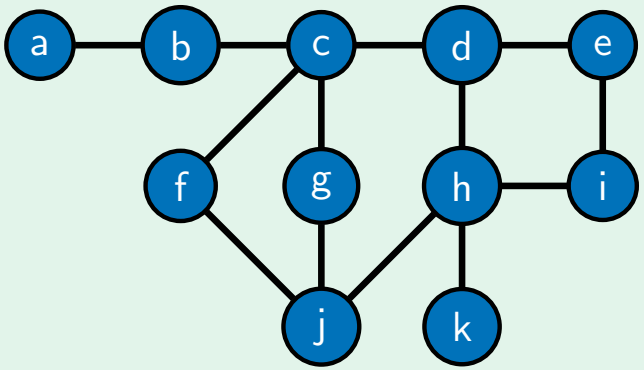
Definición

Sea $G = (V, E, \varphi)$ un grafo.

- La **excentricidad** del vértice $v \in V$ es $e(v) := \max\{d(v, z) : z \in V\}$.
- El **radio** del grafo G es $\text{rad}(G) = \min\{e(v) : v \in V\}$.
- El **diámetro** del grafo G es $\text{diam}(G) = \max\{e(v) : v \in V\}$.

Ejemplo (Excentricidad, radio y diámetro)

$G = (V, E, \varphi)$



$e(a) = 5,\ e(b) = 4,\ e(c) = 3,\ e(d) = 3$

$e(e) = 4,\ e(f) = 3,\ e(g) = 3,\ e(h) = 4$

$e(i) = 5,\ e(j) = 4,\ e(k) = 5$

$\text{rad}(G) = 3,\ \text{diam}(G) = 5$

Centro

Definición y ejemplos

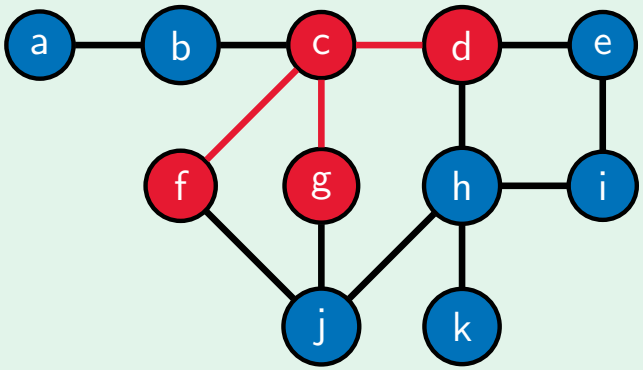
Definición

Sea G un grafo. El **centro** de G es el subgrafo inducido por el conjunto de vértices de excentricidad mínima, es decir, igual al radio.

Observaciones

- Los vértices que forman el centro de un grafo son los vértices cuya distancia máxima a todos los demás vértices es mínima.
- En el problema de ubicación de servicios, estos deberían colocarse en vértices del centro.

Ejemplo (Centro)



Centro

Vértices de excentricidad mínima: c, d, f, g.

$e(c) = e(d) = e(f) = e(g) = 3$

Centro

Caso de un árbol

Propiedad: Centro de un árbol

El centro de un árbol está formado por uno o dos vértices.

Algoritmo para obtener centro de árbol sin pesos

Input: Árbol sin pesos T.

Clave: Retirar hojas del árbol hasta llegar al centro.

Inicialización: $H = T$ árbol auxiliar y s sucesión de grados de H.

Iteración: Se realiza la siguiente comprobación:

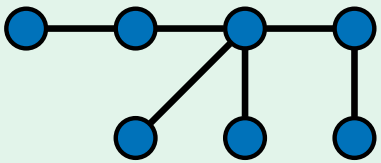
1. Si $H = K_1$ o $H = K_2$, entonces $H = C(T)$.
2. En otro caso, borramos todas las hojas del árbol auxiliar H y actualizamos la sucesión de grados s.

Output: Centro del árbol $C(T)$.

Centro

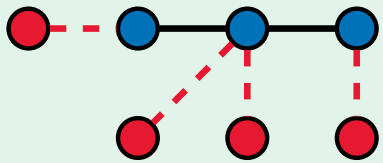
Ejemplo en el caso de un árbol

Ejemplo (Cálculo del centro de un árbol)



$s = (4, 2, 2, 1, 1, 1, 1)$

$H = T,\ H \neq K_1, K_2$



$s = (2, 1, 1)$

$H \neq K_1, K_2$

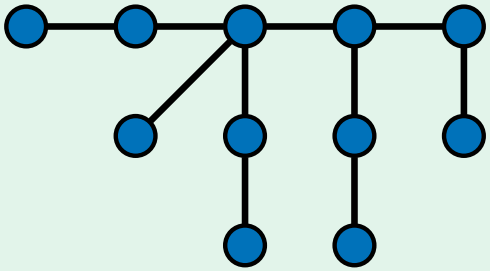


$s = (0)$

$H = K_1$

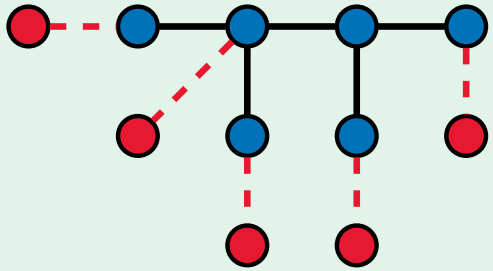
H es el **centro**

Ejemplo (Cálculo del centro de un árbol)



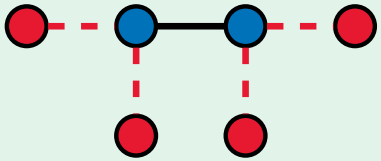
$s = (4, 3, 2, 2, 2, 2, 1, 1, 1, 1, 1)$

$H = T,\ H \neq K_1, K_2$



$s = (3, 3, 1, 1, 1, 1)$

$H \neq K_1, K_2$



$s = (1, 1)$

$H = K_2$

H es el **centro**

Periferia

Definición y ejemplos

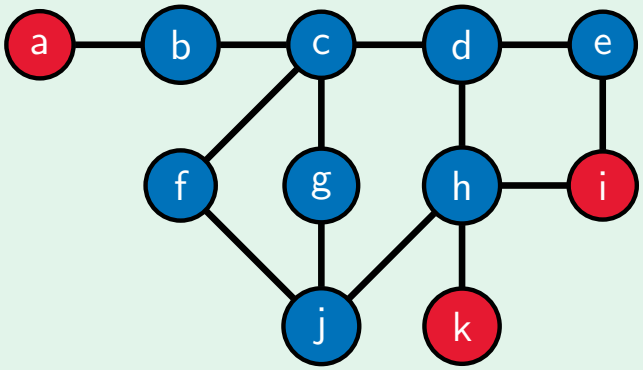
Definición

Sea G un grafo. La **periferia** de G es el subgrafo inducido por el conjunto de vértices de excentricidad máxima, es decir, igual al diámetro.

Observaciones

- Los vértices de la periferia son los vértices cuya distancia máxima a todos los demás vértices es máxima.
- En el problema de ubicación de servicios, estos son los peor ubicados para poner un servicio.

Ejemplo (Periferia)



Periferia

Vértices de excentricidad máxima: a, i, k.

$e(a) = e(i) = e(k) = 5$

Distancia total y mediana

Definición y ejemplos

Definición

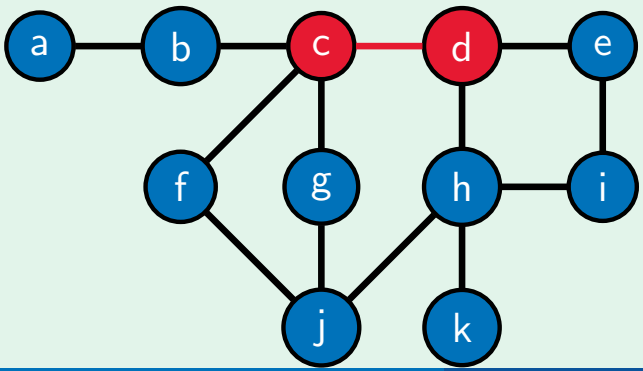
Sea $G = (V, E, \varphi)$ un grafo.

- La **distancia total** de un vértice $v \in V$ es la suma de las distancias desde ese vértice a todos los demás: $dt(v) = \sum_{z \in V} d(v, z)$
- La **mediana** del grafo G es el subgrafo inducido por los vértices de distancia total mínima.

Observaciones

En el problema de ubicación de servicios, los servicios de distribución deberían colocarse en los vértices que forman la mediana.

Ejemplo (Mediana)



$dt(a) = 34$, $dt(b) = 25$, $dt(c) = 18$, $dt(d) = 18$

$dt(e) = 25$, $dt(f) = 22$, $dt(g) = 22$, $dt(h) = 19$

$dt(i) = 26$, $dt(j) = 21$, $dt(k) = 28$

Mediana Vértices de distancia total mínima: c, d

Observaciones

- En grafos sin pesos, cada arista computa con peso igual a 1 para calcular las medidas descritas: excentricidad, radio, diámetro, centro, distancia total, mediana...
- En la mayoría de modelos donde nos interesan estas medidas los grafos son ponderados. Todas las definiciones anteriores son válidas también aquí.
- Sin embargo, la dificultad del problema aumenta para grafos ponderados.

Conectividad por vértices y aristas

- Conectividad por vértices: k-conectividad y $\kappa(G)$.
- Conectividad por aristas: k-aristoconectividad y $\lambda(G)$.

Conectividad por vértices

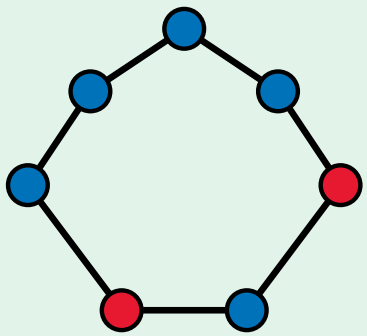
Definición

Un grafo es 2-**conexo** si cumple alguna de las siguientes condiciones:

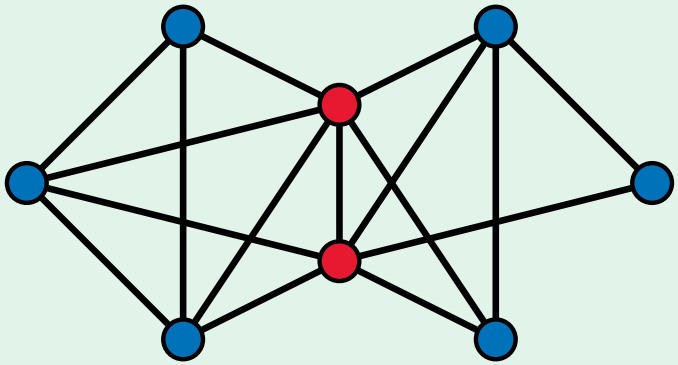
- Al eliminar cualquier vértice, el grafo sigue siendo conexo.
- Hay que eliminar al menos dos vértices para que el grafo deje de ser conexo o tenga solo un vértice.

Ejemplo (2-conexión)

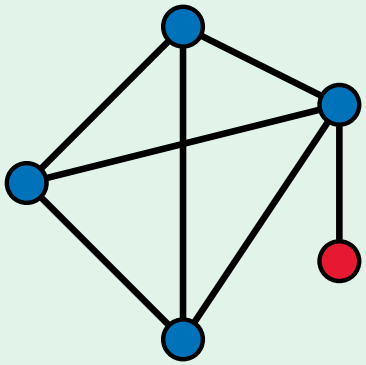
C_n es 2-conexo



Es 2-conexo



No es 2-conexo

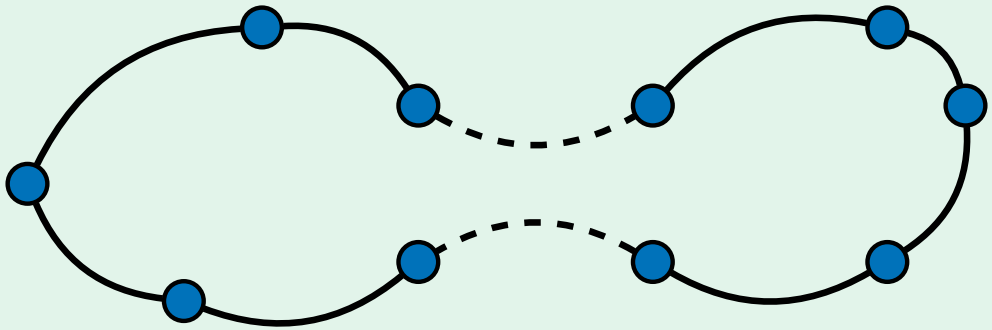


2-conectividad

Definición

Sean $G = (V, E)$ un grafo simple y $u, v \in V$ dos vértices diferentes. Decimos que dos caminos en G que van de u a v son **caminos internamente disjuntos** si no tienen ningún vértice interno común.

Ejemplo (Caminos internamente disjuntos)



Teorema (Whitney, 1932)

Sea $G = (V, E)$ un grafo simple tal que $|V| > 2$. Entonces G es un grafo 2-conexo si y solo si para cada par de vértices distintos $u, v \in V$ existen al menos dos caminos internamente disjuntos.

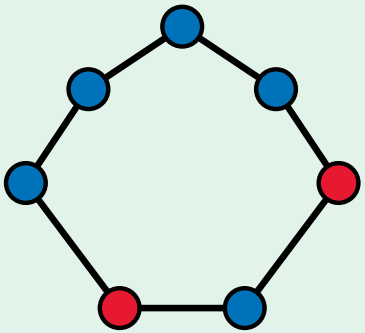
Definición

Un grafo es k-**conexo** si cumple alguna de las siguientes condiciones:

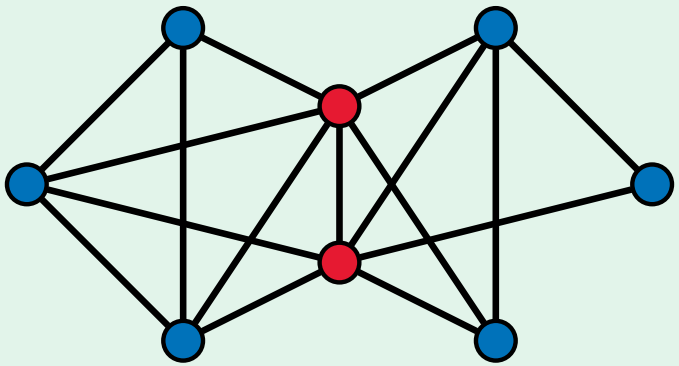
- Al eliminar cualquier conjunto de $k-1$ vértices, el grafo sigue siendo conexo.
- Hay que eliminar al menos k vértices para que el grafo deje de ser conexo o tenga solo un vértice.

Ejemplo (k-conexión)

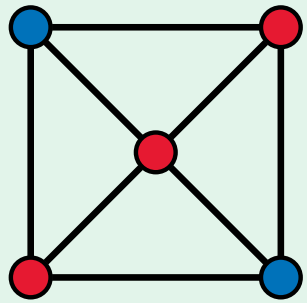
C_n es 2-conexo, pero no 3-conexo



Es 2-conexo pero no 3-conexo

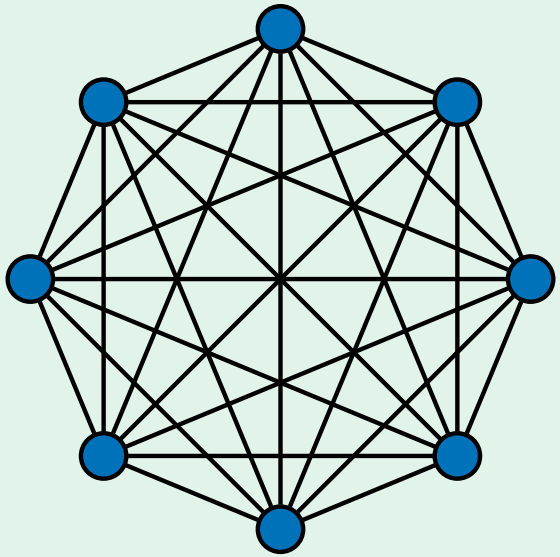


Es 3-conexo pero no 4-conexo



Ejemplo (k-conectividad)

El grafo completo K_n es $(n-1)$-conexo.



Conectividad de vértices $\kappa(G)$

Definición y propiedades

Definición

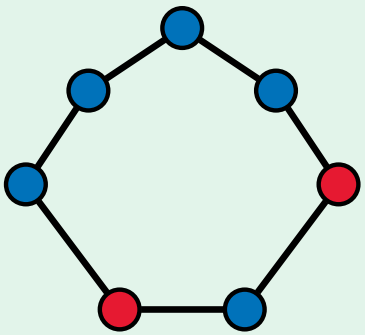
Denotamos por $\kappa(G)$ al número mínimo de vértices cuya eliminación de G produce un grafo no conexo o trivial.

Proposición

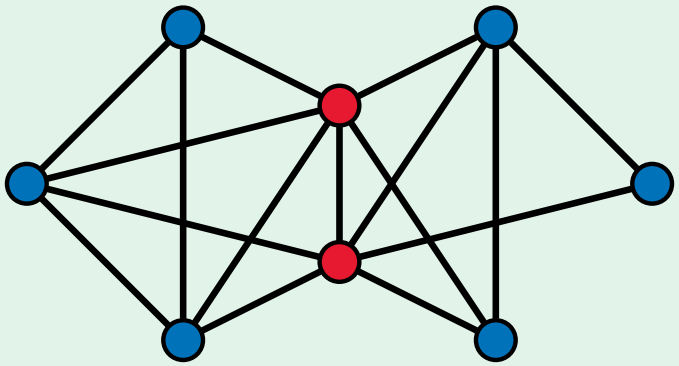
Un grafo es k-conexo si $\kappa(G) \geq k$.

Ejemplo ($\kappa(G)$)

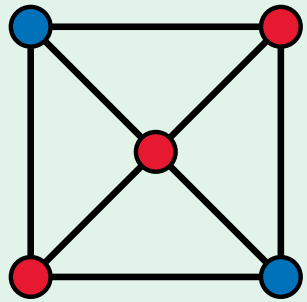
$\kappa(C_n) = 2$



$\kappa(G) = 2$



$\kappa(G) = 3$



Conectividad de vértices $\kappa(G)$

Propiedades

Proposición

Sea G un grafo y $\delta(G)$ el grado mínimo de los vértices de G.

1. $\kappa(G) \leq \delta(G)$.
2. Si G es k-conexo y tiene n vértices, tiene al menos $\lceil \frac{nk}{2} \rceil$ aristas.

Demostración.

1. Sea v un vértice de grado mínimo, es decir, $\delta(v) = \delta(G)$.
   - Si $\delta(v) = 0$, entonces v es una componente aislada: el grafo es no conexo o trivial (si no hay más vértices).
   - Si $\delta(v) \geq 1$ y eliminamos los vértices adyacentes a v, obtenemos un grafo no conexo con v como una componente conexa aislada del resto de vértices. Observamos que v tiene como mucho $\delta(G)$ vértices adyacentes. Por tanto, $\kappa(G) \leq \delta(G)$.
2. Supongamos que $G = (V, E, \varphi)$ es k-conexo. Por definición, $\delta(v) \geq \delta(G)$ para todo vértice $v \in V$. Entonces
   $2|E| = \sum_{v \in V} \delta(v) \geq \sum_{v \in V} \delta(G) = n\delta(G) \geq n\kappa(G) \geq nk \implies |E| \geq \frac{nk}{2}$

Conectividad por aristas

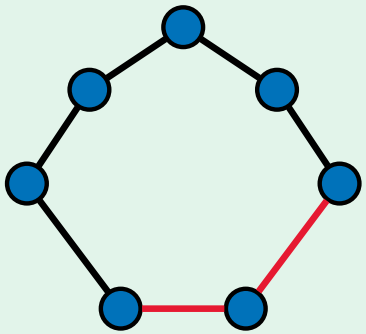
Definición

Un grafo es 2-**aristoconexo** si cumple alguna de estas condiciones:

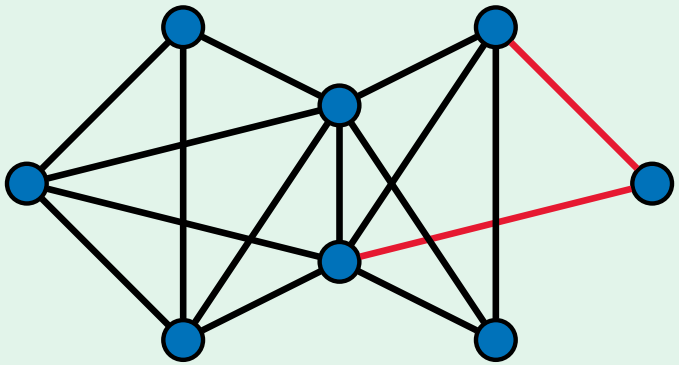
- Al eliminar cualquier arista, el grafo sigue siendo conexo.
- Hay que eliminar al menos dos aristas para que el grafo deje de ser conexo o tenga solo un vértice.

Ejemplo (2-aristoconexión)

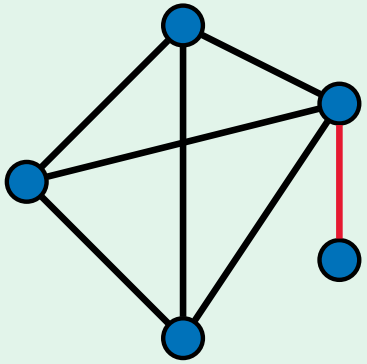
C_n es 2-aristoconexo



Es 2-aristoconexo



No es 2-aristoconexo



k-aristoconectividad

Definición y ejemplos

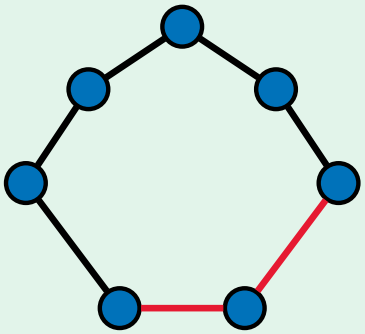
Definición

Un grafo es **k-aristoconexo** si cumple alguna de estas condiciones:

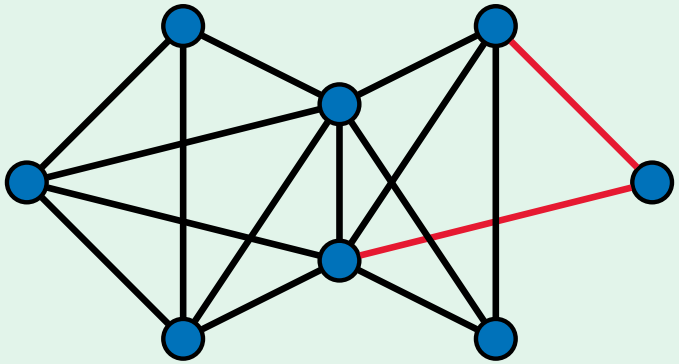
- Al eliminar cualquier conjunto de $k-1$ aristas, el grafo sigue siendo conexo.
- Hay que eliminar al menos k aristas para que el grafo deje de ser conexo o tenga solo un vértice.

Ejemplo (k-aristoconexión)

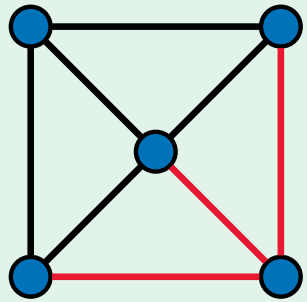
C_n es 2-aristoconexo, pero no 3-aristoconexo



Es 2-aristoconexo pero no 3-aristoconexo

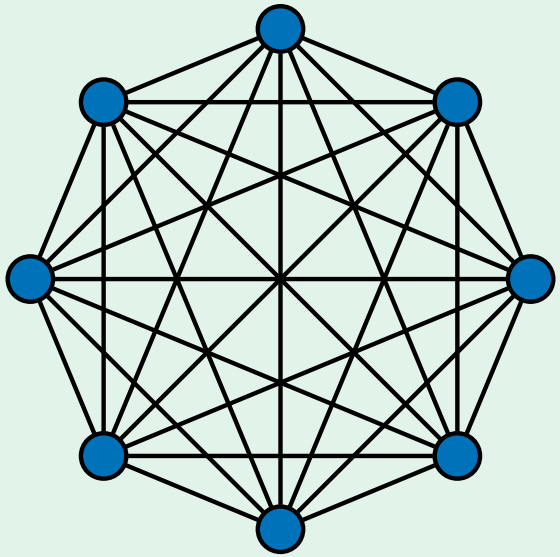


Es 3-aristoconexo pero no 4-aristoconexo



Ejemplo (k-conectividad)

El grafo completo K_n es $(n-1)$-aristoconexo.



Conectividad de aristas $\lambda(G)$

Definición y propiedades

Definición

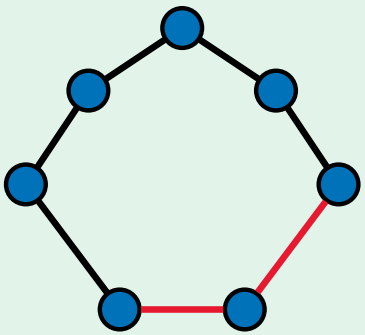
Denotamos por $\lambda(G)$ al número mínimo de aristas cuya eliminación de G produce un grafo no conexo o trivial.

Proposición

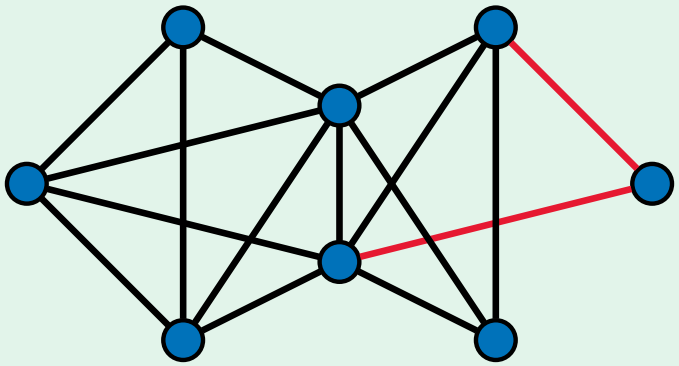
Un grafo es k-aristoconexo si $\lambda(G) \geq k$.

Ejemplo ($\lambda(G)$)

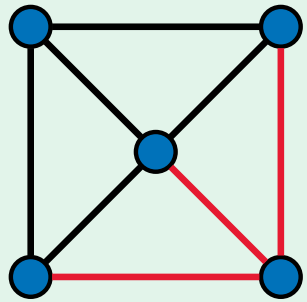
$\lambda(C_n) = 2$



$\lambda(G) = 2$



$\lambda(G) = 3$



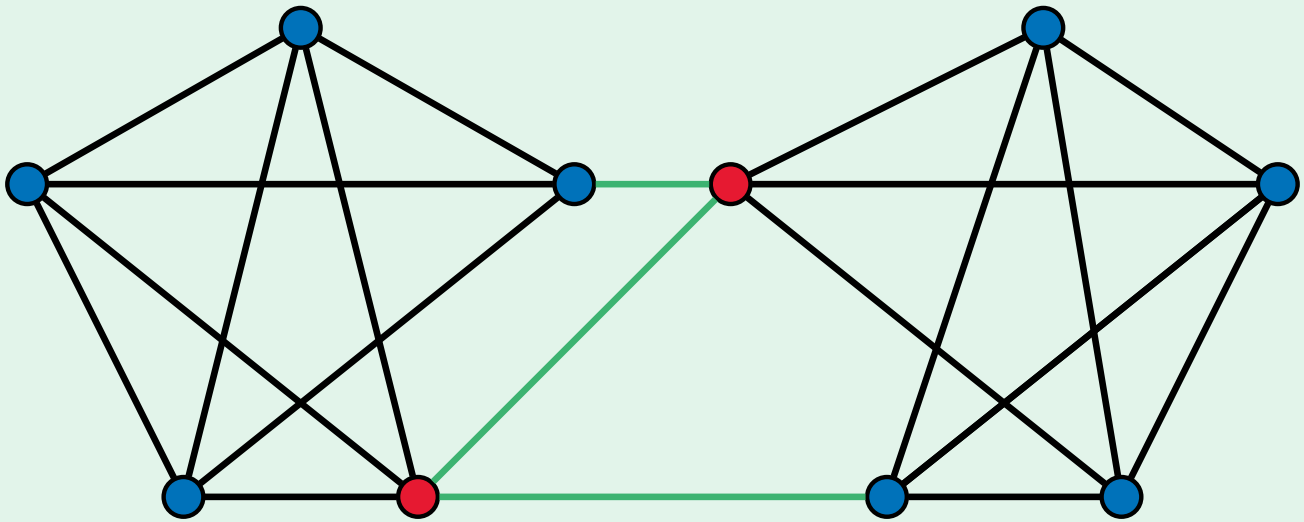
Relación entre conectividad de vértices y aristas

$\delta(G)$, $\kappa(G)$ y $\lambda(G)$

Teorema (Whitney, 1932)

Si G es un grafo simple conexo, entonces $\kappa(G) \leq \lambda(G) \leq \delta(G)$.

Ejemplo ($\delta(G)$, $\kappa(G)$, $\lambda(G)$)



$$\kappa(G) = 2 \leq \lambda(G) = 3 \leq \delta(G) = 4$$

Orientabilidad en grafos

- Grafo orientable: definición, caracterización y algoritmo para orientar grafos.

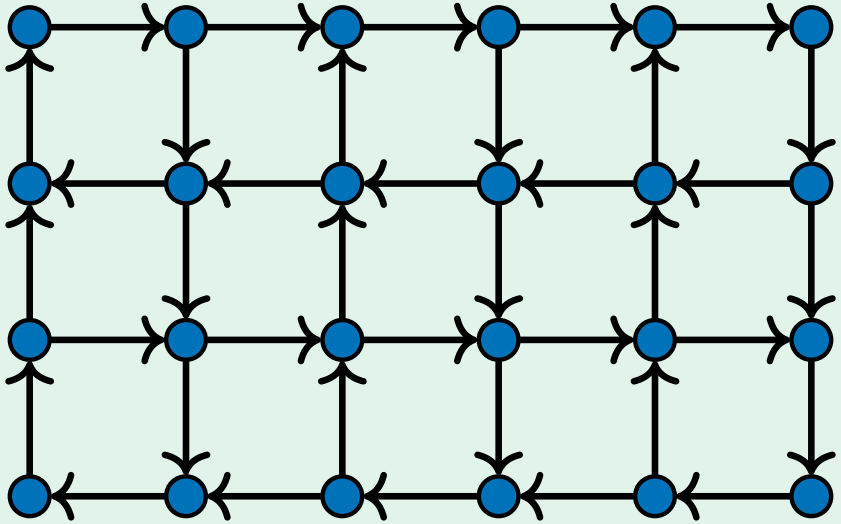
Grafo orientable

Definición y ejemplo

Definición

Un grafo es **orientable** si podemos asignar un sentido a cada una de sus aristas de forma que el digrafo resultante sea fuertemente conexo.

Ejemplo (Grafo orientable)



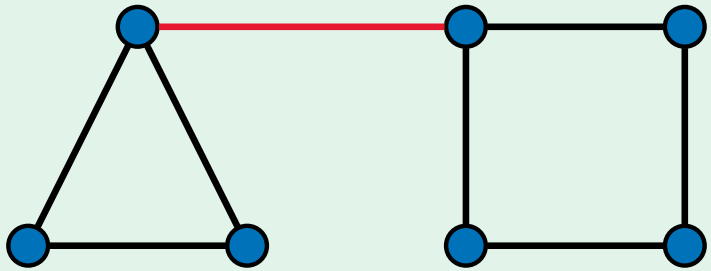
Grafo orientable

Caracterización

Teorema de Robbins

Un grafo es orientable si y solo si es conexo y no tiene puentes, equivalentemente, si es 2-aristoconexo.

Ejemplo (Grafo no orientable)



Tiene una **arista puente**

El grafo no es orientable

Algoritmo de orientación para grafo orientable

Input: Grafo 2-aristoconexo.

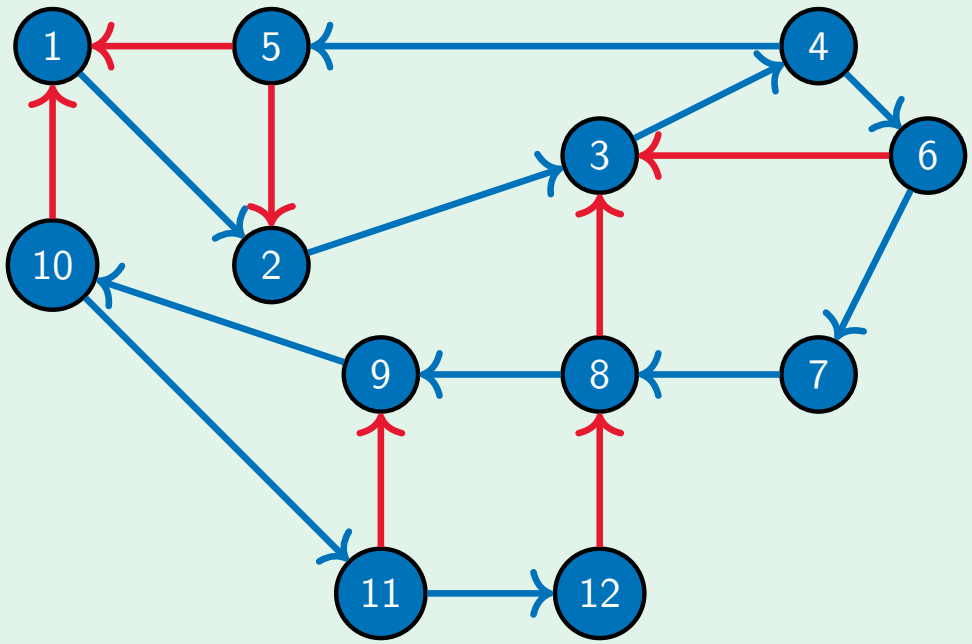
Inicialización: v_0 raíz del árbol.

Clave: Se construye el árbol de búsqueda en profundidad:

1. Cada arista que se añade al árbol se orienta de la raíz a las hojas, es decir, desde el vértice activo hasta el vértice visitado.
2. Las aristas que no se añaden al árbol se orientan de las hojas a la raíz, es decir, desde el vértice activo hasta el vértice adyacente que ya había sido visitado previamente.

Output: Grafo orientado.

Ejemplo (Orientación de un grafo orientable)



Aristas no pertenecientes al árbol de BEP

Aristas pertenecientes al árbol de BEP